从 OxyGen 出发，七天打通模型推理 Infra

2026-10-15

目录

一条请求的一生：先画出推理 Infra 的全景地图 2

七天能走到哪里：三个能力层级与诚实的边界 5

开工前的资源账：4×H800、版本冲突、模型与磁盘 6

Day 1：文本先变成 token，再穿过 36 层 Transformer 9

Day 2：prefill 吃算力，decode 吃带宽，KV cache 吃显存 12

Day 3：logits 到 token 之间还有七道工序 17

Day 4：调度器决定谁先算，KV 管理决定能装多少 19

Day 5：引擎是一组进程，服务质量要用 SLO 来衡量 23

Day 6：H800 的 400 GB/s NVLink 改写了并行选型 26

Day 7：像素、帧和波形最终都变成 prefill token 30

自检题：每一题都应能用数字或代码回答 33

结论 34

参考资料 34

附录：代码仓库、文件与工具链接清单 36

七天时间足够你把模型推理 Infra 的主干从头到尾走一遍，并在 4×H800 上做出一套能复现的对照实验，但到不了生产级。这条主干是：文本、像素或波形先变成 token；prefill 一次性处理整段输入并建立 KV cache；decode 循环里，调度器把这个请求和其他请求合成一批；最后经过采样和流式反分词送回用户。下面按七个连续的日子安排：

- Day 1：预处理与 Transformer 前向
- Day 2：prefill/decode、KV cache 与 roofline
- Day 3：采样、停止条件与结构化输出
- Day 4：continuous batching、PagedAttention 与 prefix caching
- Day 5：引擎架构、服务指标、speculative decoding 与 PD 分离
- Day 6：并行、通信、profiling 与量化
- Day 7：多模态数据通路，最后回到 VLA

每天都有一节 OxyGen 对照。你已经复现的三件事——同一帧只 prefill 一次、动作专家与语言专家共享 KV、语言解码跨帧 continuous batching——分别对应 prefix 共享和 iteration-level scheduling 在机器人场景里的特化形式。不同之处有三点：迭代粒度是”帧”，KV 是静态预留而不是分页，前缀用的是双向注意力。

贯穿七天的物理事实只有一条：**decode 每一步都要把全部权重和本序列的 KV 从 HBM 读一遍。**以 Qwen3-8B 为例，在单张 H800 上、batch=1 时，这条约束给出的下限约为 **4.5 ms/token (约 220 tok/s)**。batching、调度、量化、并行，本质上都是在这条约束下想办法。

全部实验约需 **60 GPU·h**，只做推理，不做训练。配套的 10 个 labs 已在 CPU 和 RTX 4070 Ti SUPER 上通过自检，但**没有在 H800 上跑过**，所以文中所有 H800 数字都要你自己实测。

版本基线为 vLLM 0.31.0、SGLang 0.5.21、transformers 5.19.0。事实来自研究笔记中 2026-10-14/15 访问的官方源码、文档和论文。NVIDIA 的 H800 官方规格页已经返回 404，硬件数字按来源分级，并标注「未核实」。

阅读约定：

- 「未核实」：研究笔记没有拿到一手来源。
- 「证据不足」：来源互相矛盾，或抓取失败。
- 「推断」：根据已引用的事实计算或推理得出。
- 验收条目前的 [概念] 表示能讲清、能手算；[实验] 表示能跑出数据并用指标解释。
- labs 用相对路径`../labs/...`引用。代码摘录逐字取自文件。
- 正文链接用描述性文字。文中提到的每个代码仓库、源码文件、工具、文档页、模型卡和数据集，完整 URL 都写在文末的「附录：代码仓库、文件与工具链接清单」中。
- 另有一份面向 agent 的环境搭建说明`../AGENT_SETUP.md`和配套脚本目录`../scripts/`。其中的命令取自与本文相同的研究笔记，脚本只做过语法检查，没有在 H800 上运行。

一条请求的一生：先画出推理 Infra 的全景地图

学推理 Infra，最容易犯的错是先钻进某个优化技巧（PagedAttention、FlashAttention、TP），却不知道它作用在请求生命周期的哪一段、在跟哪种资源较劲。所以第一件事是把一条请求从进入到离开的完整路径画出来。下图以 vLLM V1 为骨架，SGLang 的结构基本可以一一对应。

```
client (OpenAI-compatible HTTP, SSE stream)
   |
   v
[1] API server process ------------------------------------------+
    chat template -> tokenizer -> input_ids                      |
    multimodal: decode/resize/normalize -> processor cache       |
   |  ZMQ                                                        |
   v                                                             |
[2] Engine core: scheduler busy loop (every iteration)           |
    waiting queue -> prefix cache lookup (block hash / radix)    |
    allocate KV blocks -> token budget (running first, then      |
    chunked prefill of waiting) -> preempt (recompute) if full   |
```

```
   |                                                                  |
   v                                                                  |
[3] GPU worker / model runner (one per GPU, TP x PP)                  |
    build input ids, positions, block tables -> CUDA graph            |
    embed -> L x [RMSNorm, QKV, RoPE, paged attn, O, SwiGLU]          |
    -> final norm -> lm_head (only positions to be sampled)           |
   |                                                                  |
   v                                                                  |
[4] Sampler: fp32 logits -> penalties -> temperature                  |
    -> min_p / top_k / top_p -> exponential-race sampling             |
    (+ structured-output bitmask, logprobs)                           |
   |                                                                  |
   v                                                                  |
[5] Output processor: incremental detokenize, stop strings,           |
    metrics (TTFT, ITL, TPOT) -> stream text back <----------------+
   loop [2]..[5] until EOS / stop / max_tokens
```

[1] 入口：API server 进程。这一层负责 HTTP、tokenize 和多模态加载，通过 ZMQ 与 engine core 通信。默认只有 1 个 API server；开数据并行（DP）时会自动扩展到 DP 个（vLLM arch_overview @v0.31.0）。

以 Qwen3 为例，这一步的具体内容如下：

- chat template 是一段 Jinja 模板，把 messages 渲染成带 `<|im_start|>` 的字符串。`add_generation_prompt` 会在末尾补上 `<|im_start|>assistant\n`。
- tokenizer 依次做 NFC 规范化、正则切分和 byte-level BPE。Qwen3 不加 BOS，`eos_token` 为 `<|im_end|>`（Qwen3-0.6B tokenizer_config.json）。
- 输出是 `input_ids`。HF 的静态 batch 还会带上 `attention_mask`。

多模态请求在这一层还要额外做 CPU 上的解码、resize 和归一化。处理结果按 blake3 内容哈希存进 processor cache，默认 4 GiB（vllm/config/multimodal.py @v0.31.0）。

[2] 调度：Engine core。这是整个系统的心脏。vLLM V1 调度器的源码注释写得很直白：“There’s no ‘decoding phase’ nor ‘prefill phase’ in the scheduler”。每个请求只有两个数：`num_computed_tokens` 和 `num_tokens_with_spec`，调度就是让前者追上后者。chunked prefill、prefix caching 和 speculative decoding 都能用这一个抽象统一表达（scheduler.py @v0.31.0）。

每一步的流程是：

1. 先给 RUNNING 请求排工作，所以 decode 优先。
2. 再用剩余的 token 预算接纳 WAITING 请求，长 prompt 在这里被切块（chunked prefill）。
3. 如果 KV block 不够，就做 **recompute 式抢占**：被抢占的请求释放全部 block，`num_computed_tokens` 归零，回到等待队列队首。scheduler.py 里没有 swap 路径。

接纳新请求之前，会先查 prefix cache：vLLM 用满块的链式哈希，SGLang 用 radix tree。命中的部分不必重新计算。

[3] 执行：GPU worker。每张 GPU 一个 worker 进程，数量为 TP×PP。所以 `vllm serve -tp=4` 共有 1 个 API server、1 个 core 和 4 个 worker，合计 6 个进程；`-tp=2 -dp=4` 则是 17 个（arch_overview @v0.31.0）。

worker 的工作分三步：

- 把调度结果组装成输入张量和 block table。
- 在 uniform decode 时 replay 一张完整的 CUDA graph。v0.31.0 默认 `FULL_AND_PIECEWISE`（cuda_graphs.md @v0.31.0）。
- 执行 Transformer 前向。

在 CUDA 上，满足条件时默认使用 Model Runner V2，它的设计是“Async-First”：GPU 在执行第 N 步的同时，CPU 已经在准备第 N+1 步（model_runner_v2.md @v0.31.0）。

[4] 采样。logits 先转成 fp32，然后依次经过：白名单、bad words、min_tokens/logit_bias、三种 penalty、temperature、min_p、top-k/top-p，最后采样。默认返回的 logprobs 是在**原始 logits** 上计算的。采样时

不调用 `torch.multinomial`，而是用 `argmax(p/q)`，其中 q~Exp(1)，以避免 CPU-GPU 同步（sampler.py @v0.31.0、topk_topp_sampler.py）。

[5] 输出。增量反分词时，遇到不完整的 UTF-8 字节先缓冲，凑齐再输出。stop 字符串在 detokenizer 层用尾部缓冲判断（vllm/v1/engine/detokenizer.py）。TTFT、ITL 等指标在这一层记录。

SGLang 的对应结构是：

- HTTP server 加 TokenizerManager，相当于 [1]。
- 每个 TP rank 一个 Scheduler 进程，进程内含 TpModelWorker，相当于把 vLLM 的 [2] 和 [3] 合在一起。
- DetokenizerManager，相当于 [5]。

SGLang 的 `event_loop_overlap` 会先把当前 batch 发射到 GPU，再去处理上一个 batch 的结果，让 CPU 调度和 GPU 前向重叠（SGLang scheduler.py @v0.5.21）。

其他模态只是在同一张地图上加站点

其他模态没有另起一套系统，只是在上面的地图上加了几个站点：

- **VLM**：在 [1] 增加 CPU 预处理，在 [3] 之前增加视觉编码器和 merger。然后用占位 token 加 `masked_scatter`，把视觉 embedding 写进文本序列，之后的 prefill 与纯文本完全相同（modeling_qwen3_vl.py @v5.19.0）。
- **音频输入**：多一个 Mel 前端和音频编码器。
- **视频**：本质上是带时间维的图像，token数常常爆炸。
- **Omni 语音输出**：变成 Thinker、Talker、Code2Wav 三个阶段的流水线。
- **扩散生成**：没有随长度增长的 KV cache，主成本是”步数 × 每步全序列计算”。
- **VLA**：前半段就是 VLM 的 prefill，后半段把 LLM 解码换成（或并联上）动作专家。

Day 7 会把这些路径逐一量化。

你的 OxyGen 落在地图的哪里

OxyGen 每一帧的数据流如下：

```
frame t: new observation o_t
  [prefill ONCE] SigLIP 3 x 256 image tokens + text/state tokens
                 -> Gemma-2B (bidirectional prefix) -> prefix KV
        |                                    |
        v                                    v
  action expert (gemma_300m)          new language state sigma_t
  S = 10 flow-matching steps,         joins active set R
  each reads prefix KV, no append           |
        |                                    v
  action chunk A_t (H = 10)           batched decode: k tokens for all
                                      m requests in R; finished evicted,
                                      the rest saved for frame t+1
```

OxyGen 论文 v3 的做法是：每一帧对新观测只做一次 prefill，得到的 KV 同时交给动作专家（S 步 denoising）和一个新建的语言状态；然后把所有活跃语言请求拼成一批，推进 k 步（OxyGen arXiv 2603.14371 v3）。论文明确说明，在 LLM server 里简单打开 prefix caching 和 continuous batching “does not provide this coordination”，因为同一份 prefill 要服务两个**权重不同**的专家，而且语言解码必须塞进周期性动作推理的时间预算里。下表把它和通用 serving 概念对齐，类比失效的地方单独列出：

OxyGen 环节	通用推理概念	类比在哪里失效
图像 resize + 状态离散化 + 分词	预处理 / tokenize	π0.5 把状态写进 prompt，每帧都变

OxyGen 环节	通用推理概念	类比在哪里失效
SigLIP + Gemma-2B 前缀前向	prefill（决定 TTFT）	前缀双向注意力，无法跨帧精确复用
10 步 flow matching	多次小前向复用同一 KV	非自回归、不追加 KV、步数固定
语言 token / π0-FAST token	decode（带宽受限）	FAST 用 2B 全骨干解码动作
一次 prefill 供两个专家	prefix 共享（ref_cnt / radix）	同帧跨专家共享，不是跨请求共享
每帧推进 k 个 token	iteration-level scheduling	粒度是帧，KV 静态预留不分页
动作逐帧截止期	per-iteration 延迟 SLO	H/T 是吞吐型指标，不是反应时延

表中事实来自 openpi pi0.py @215abfb 和 OxyGen policy.py @1e88906。

还有一个关于接收状态的事实需要说明：OxyGen 的 NeurIPS 2026 接收状态只来自作者自述，即 arXiv v3 页眉和 README。研究笔记没能在 neurips.cc 或 OpenReview 上独立核实。

七天能走到哪里：三个能力层级与诚实的边界

这份指南没有因为”只有七天”去删核心内容，而是把七天的产出说清楚：**概念掌握可以完全达成，可运行实验是核心交付，生产级能力达不到。**下表的分级是笔者基于官方文档的能力项做的推断，不是引用某个权威标准。

层级	标志性能力	可检验产出	7 天内
概念掌握	讲清并手算 KV、roofline、调度与通信量	一页架构图和手算表	完全可达
源码定位（概念的延伸）	在 nano-vllm / mini-sglang 中定位并小改	调用链笔记和一个小 patch	可达
可运行实验	固定版本做受控对照，用指标解释	曲线、原始 JSON、复现脚本	核心交付
工程调优（实验的延伸）	用 profiler 证据选并行与 batch 上限	1–2 个带 trace 的调优点	部分可达
生产级能力	SLO 容量规划、可观测、扩缩容、恢复	线上服务、runbook、回归 CI	不可达

生产级能力缺的不是”再多几个参数”，而是真实流量、集群和数周到数月的工程积累。具体包括以下几项：

- **按 goodput 做容量规划**：goodput 指满足 SLO 的吞吐，是 DistServe 的优化目标（DistServe arXiv 2401.09670）。
- **告警**：基于 `vllm:num_requests_waiting`、`vllm:kv_cache_usage_perc`、TTFT/ITL 直方图等指标（vLLM metrics design）。
- **自动扩缩容**：以队列长度或 batch size 为信号。GKE 文档建议队列阈值从 3–5 开始调，并说明 GPU 内存或 CPU 指标单独使用不够（GKE autoscaling best practices）。
- **K8s 上的分布式推理栈**：多租户流控、KV 感知路由和 PD 分离，代表项目有 llm-d、NVIDIA Dynamo、KServe、vLLM production-stack。
- **冷启动与权重分发**；**故障转移与请求迁移**，学术代表是 Llumnix arXiv 2406.03243。
- **每个版本的回归基准**、**$/Mtoken成本核算**，以及**量化后的任务级精度评测。**

Dynamo README 中的”TTFT 快 2 倍” ”冷启动快 7 倍”等数字是厂商自述。

时间假设：每天 10–12 小时，大约是理论 3–4 小时、实验 4–5 小时、读代码 2–3 小时。GPU 实验大多可以并行，或放到后台跑，让人和机器同时满载。如果每天只有 8 小时，就把标为”可选”的条目整体顺延到第二周，核心条目一条都不要删。七天结束时，你应当拥有：

- 一个装满固定版本、原始数据和曲线的个人基准目录；
- 一张能默画出来的请求生命周期图；
- 一份把 OxyGen 用通用 Infra 语言重新解释的笔记。

开工前的资源账：4×H800、版本冲突、模型与磁盘

先确认你手上的 H800 到底是什么

NVIDIA 的 H800 产品页（美国站和中国站）在 2026-10-14 访问时都返回 404，所以 H800 的参数只能从其他来源拼出来（nvidia.com h800）。可靠的结论是：**H800 SXM 的 Tensor Core 算力和 HBM 与 H100 SXM 相同，被削减的是 NVLink（900→400 GB/s）和 FP64。**

指标	H100 SXM（T1）	H800 SXM5	H800 来源等级
BF16 dense	989 TF	989 TF	T2 腾讯云
FP8 dense	1,979 TF	约 1,979 TF	推断
HBM 容量/带宽	80 GB / 3.35 TB/s	80 GB / 3.35 TB/s	T3，「未核实」
NVLink	900 GB/s	400 GB/s	T1 DeepSeek
SM 数	132	132	T1 DeepSeek

各项来源：

- H100 的数字来自 NVIDIA H100 产品页。页面上带 * 的数字含 2:4 稀疏，dense 值是它的一半。
- H800 的 NVLink 和 SM 数来自 DeepSeek 的两篇论文：原文是 “reduced from 900 GB/s to 400 GB/s”，单向约 200 GB/s、实际可达约 160 GB/s；用于通信的 SM 写的是 “20 out of the 132 SMs”（arXiv 2505.09343、arXiv 2412.19437）。
- BF16 989 TF 来自 腾讯云 HCCPNV5 规格。
- 3.35 TB/s 只有经销商页面给出（强川科技 H800 页面），而且同一页面的宣传文字又写成”3 TB/s”，自相矛盾，所以标为「证据不足」。一条旁证是：FlashMLA 在 H800 SXM5 上实测达到 3000 GB/s，说明有效带宽至少有 3 TB/s（FlashMLA @ba89a34）。

本指南做规划时取 3.35 TB/s 作为理论上限，Day 2 会用实测数据替换它。

如果你的卡是 **H800 PCIe**，情况完全不同：HBM2e、2 TB/s、BF16 dense 约 756 TF，NVLink 只能用 bridge 两两相连（Lenovo Press LP1814）。这时跨对做 TP4 会走 PCIe，Day 6 的所有并行结论都要重算。所以第一条命令必须先看拓扑：

```
nvidia-smi            # 记录驱动版本；SGLang 0.5.21 需要支持 CUDA 13 的驱动
nvidia-smi topo -m    # 4 卡两两之间是 NV#（NVLink）还是 PIX/PXB/PHB/SYS
```

图例中，`NV#` 表示“traversing a bonded set of # NVLinks”，`SYS` 表示跨 socket 互连（nvidia-smi 文档）。

版本与环境：先处理冲突，默认值其次

下面五处冲突必须在开工前处理，而不是等到报错时再查。

1. **vLLM 默认 wheel 的 CUDA 版本：「证据不足」。**来源互相矛盾：
   - v0.31.0 release notes 写的是 “PyPI (CUDA 13.0)”，Docker `vllm/vllm-openai:v0.31.0` 为 CUDA 13.0，另有 `-cu129` 标签（vLLM v0.31.0 release）。
   - 同一 tag 下的安装文档写的是 “binaries are compiled with CUDA 12.9 ...by default”（gpu.cuda.inc.md @v0.31.0）。

- 最新文档也写 12.9 是默认，另有 12.8/13.0/13.1 变体（vLLM GPU install）。

做法：装好后用 `python -c "import torch;print(torch.version.cuda)"` 确认实际版本。

2. **Python 版本说法有三种。** PyPI 写 `>=3.10,<3.15`（PyPI vllm），tag 文档写 3.10–3.13，最新文档写 3.11–3.14。SGLang 要求 Python ≥3.11。所以**统一用 Python 3.12**，三种说法都满足。
3. **SGLang 0.5.21 只支持 CUDA 13。**文档原文是“SGLang 0.5.19 is the last release with a CUDA 12 lane”（SGLang install.mdx @v0.5.21）。文档说 cu129 是”随 PyTorch 2.14 升级”退役的，但 PyPI 上 0.5.21 实际固定的是 `torch==2.13.0`（PyPI sglang），这一点「证据不足」。CUDA 13.x 要求驱动 ≥580（R580），12.9 GA 要求驱动 ≥575.51.03（CUDA Toolkit Release Notes）。**想在同一台机器上对比两个引擎，驱动应 ≥580。**
4. **两个引擎的依赖无法共存。** vLLM 0.31.0固定 `flashinfer-python==0.7.0.post1`、`transformers>=5.10.4,<5.18.0`、`xgrammar==0.2.7`；SGLang 0.5.21 固定 `flashinfer_python[cu13]==0.6.18`。所以 **vLLM、SGLang 和 transformers 5.19 实验应分别放进三个独立的 venv 或容器。**注意，本文的 HF 源码事实（例如 v5 已移除 `cache_position`）是在 transformers 5.19.0 上读的，5.17 是否一致「未核实」。
5. **FlashMLA 的 main 分支已在 2026-09-30 移除 Hopper 支持**，现在要求 SM100 和 CUDA 13.1+。H800 用户必须固定在 commit `ba89a34`（FlashMLA README）。DeepEP 的 V2/V2.5 要求 CUDA 13.1+ 和 PyTorch 2.10+（DeepEP README）。

最稳妥的做法是用钉死 tag 的官方镜像，命令取自笔记中的官方文档：

```
docker pull vllm/vllm-openai:v0.31.0          # CUDA 13.0（据 release notes）
docker pull vllm/vllm-openai:v0.31.0-cu129    # 驱动只支持 CUDA 12.x 时用
docker pull lmsysorg/sglang:v0.5.21           # 仅 CUDA 13
# 不用容器时（各自独立 venv）：
uv pip install vllm --torch-backend=auto
uv pip install --prerelease=allow sglang==0.5.21
```

模型、磁盘与 GPU 的七天分配

模型全部为开放权重，不需要 Llama 的 gated 权限。bf16 体积按”参数量 × 2 字节”推算，不是实测的磁盘占用（HF API Qwen3-8B）。

模型	bf16 体积	用于	必需性
Qwen/Qwen3-0.6B	约 1.5 GB	Day 1–5 读源码、小实验	必需
Qwen/Qwen3-8B	约 16.4 GB	Day 2–6 单卡基准	必需
Qwen/Qwen3-32B	约 65.5 GB	Day 6 TP 对比	必需
Qwen/Qwen3-30B-A3B	约 61 GB	Day 6 MoE / EP	必需
Qwen/Qwen2.5-VL-7B-Instruct	约 16.6 GB	Day 7 VLM	必需
Qwen/Qwen3-VL-8B-Instruct	约 17.5 GB	Day 7 VLM 对照	建议
openai/whisper-large-v3	约 3.1 GB	Day 7 音频	可选
Qwen/Qwen2.5-VL-72B-Instruct	约 145 GB	Day 7 encoder TP	可选

必需和建议的模型合计约 180 GB，加上可选项约 330 GB；再算上镜像和 venv，**建议在本地 NVMe 上预留 0.6–1 TB。**用 `hf` CLI 下载（旧名 `huggingface-cli` 已改名），默认缓存在 `~/.cache/huggingface/hub`，由 `HF_HOME` 控制（HF CLI guide）：

```
export HF_HOME=/data/hf        # 放在大容量 NVMe 上，并挂载进容器
hf download Qwen/Qwen3-0.6B
hf download Qwen/Qwen3-8B
hf download Qwen/Qwen3-32B
```

```
hf download Qwen/Qwen3-30B-A3B
hf download Qwen/Qwen2.5-VL-7B-Instruct
hf download Qwen/Qwen3-8B config.json --local-dir ./cfg/qwen3-8b   # 给 lab08 用
```

如果想让 agent 代为搭建环境，可以把`../AGENT_SETUP.md`交给它，按其中的步骤依次运行`../scripts/00_check_env.sh`到`08_sglang_serve_bench.sh`；统一配置只在`../scripts/config.sh`中修改。命令若与官方文档的当天版本不一致，以官方文档为准。

如果下载或`docker pull`失败，只记录报错信息和失败的是哪一步。按照你的分工，网络和代理问题交给 Codex 排查；你先继续做不依赖网络的 CPU labs。

七天的 GPU 预算：

天	主题	主要用卡	约 GPU·h	关键产出
Day 1	预处理与前向	1 卡	2	形状表、logits 对齐
Day 2	KV 与 roofline	4 卡分任务	7	实测 vs 理论表
Day 3	采样与结构化	4 卡分任务	5	非确定性与 TPOT 表
Day 4	批处理与 KV 管理	4 卡分任务	9	并发曲线、缓存表
Day 5	引擎与服务	4 卡分任务	10	goodput 曲线
Day 6	并行与 profiling	4 卡整机	20	并行选型表、trace
Day 7	多模态与 VLA	4 卡分任务	10	TTFT–token 曲线

合计约 63 GPU·h，只占 4×H800 一周可用卡时（672）的一成左右。剩余卡时留给重跑和扩展；这份计划不需要做任何规模的训练。

配套 labs 的真实状态

10 个 labs 都放在`../labs/`下，使用随机权重或合成数据，不需要下载任何东西。每个 lab 都带`--test`自检模式，逐条打印 PASS/FAIL。

本机测试环境有两个：

- CPU 环境：Python 3.11、torch 2.13.0+cpu。
- GPU 环境：Python 3.10、torch 2.5.1+cu121、transformers 5.13.0、RTX 4070 Ti SUPER 16 GB。

研究笔记 code_labs.md 记录了最终一轮完整验证，所有命令的退出码都是 0。

lab	讲什么	本机实测	未在本机运行
lab01	字节级 BPE、流式反分词	8/8 PASS	—
lab02	最小 Llama/Qwen3、KV cache	CPU/CUDA 4/4	H800 计时
lab03	HF 权重映射、logits 对齐	CPU/CUDA 4/4	`--pretrained <hf_id>`
lab04	惩罚、温度、top-k/p、min-p	CPU/CUDA 16/16	—
lab05	分页 KV、前缀缓存、LRU	CPU/CUDA 9/9	—
lab06	调度模拟、抢占重算	13/13	真实引擎对照
lab07	TP 块、all_reduce	gloo 2 进程 2/2	torchrun、NCCL
lab08	容量与 roofline 计算	13/13	真实 config + H800

lab	讲什么	本机实测	未在本机运行
lab09	profiler、trace、NVTX	CPU 3/3，CUDA 3/3+1 SKIP	nsys、GPU kernel 轨迹
lab10	图像 patch/合并、Mel	CPU/CUDA 11/11	—

需要特别说明的局限：

- **Windows 上的 torchrun 不可用。** torch 2.13 和 2.5.1 在 rendezvous 阶段都会报 libuv 错误，所以 lab07 提供了自带的 `--spawn` 启动器。
- **本机抓不到 GPU kernel 轨迹。** CUPTI 不可用，`supported_activities()` 只返回 CPU。lab09 对此打印警告并标为 SKIP，而不是 PASS。
- **NCCL、nsys 和任何 H800 数字都没有在本机运行过。**
- **玩具模型的计时只反映 Python 和 kernel 启动开销。**例如在 4070 Ti SUPER 上，lab02 的 decode 反而比 CPU 慢，这些数字不能代表 H800 的性能（见../labs/README.md）。

Day 1：文本先变成 token，再穿过 36 层 Transformer

目标：把一条 chat 请求精确追踪到 `input_ids`；能写出 Qwen3-8B 一层里每个算子的输入输出形状；能手算参数量和每 token 的 KV 字节数；在 H800 服务器上把 lab01–03 跑通。当天还要搭好环境，并在后台开始下载模型。

理论：tokenizer 是确定性流水线，Transformer 层是固定的十一步

Tokenizer。 Qwen3 的 tokenizer 是一条确定性流水线，配置都写在 `tokenizer.json` 里（Qwen3-0.6B tokenizer.json）：

1. 规范化：`NFC`。
2. 预切分：先用一条正则 `Split`，再做 `ByteLevel`。正则对数字逐位切分（`\p{N}`），而 Llama 3 是最多 3 位一组（`\p{N}{1,3}`）。
3. BPE：`byte_fallback: false`。词表 151,643 项、merges 151,387 条，另有 26 个 added tokens。

模型 config 里 `vocab_size` 是 151,936，比 tokenizer 实际的 151,669 大，说明 embedding 和 LM head 的末尾有填充行（Qwen3-0.6B config.json）。

SentencePiece 系（Llama 2、Gemma）的做法不同：把空格转义成 ▁，词表外的字符用 `<0xE9>` 这类字节 token 兜底（SentencePiece README）。

中文 token 率。 Qwen 官方的经验值是 1 token ≈ 1.5–1.8 个中文字符（Qwen Docs: Key Concepts）。研究笔记作者用纯 Python 复现了 Qwen3 的 BPE，对一段 181 字符的中文实测为 1.645 字/token。这个数可复现，但不是官方数字。生僻字"龘"（UTF-8 `e9 be 98`）会被切成两个 token，而且每个 token 单独都不是合法的 UTF-8。这正是流式输出必须先缓冲字节的原因。

chat template 与生成配置。这一层决定了 prompt 的确切字节：

- Qwen3 的 chat template 在 `enable_thinking=False` 时会预填一个空的 `<think>` 块。kwarg 如何透传到模板「未核实」，以实际输出为准（tokenizer_config.json）。
- `generation_config.json` 给的默认值是 `temperature 0.6, top_k 20, top_p 0.95`，eos 为 `[151645, 151643]`（generation_config.json）。

padding。HF 文档要求 decoder-only 模型做批量生成时用 left padding，理由是 "a LLM is not trained to continue generation from padding tokens"（llm_tutorial.md）。vLLM 这类按 token 拼接（packed/varlen）的引擎根本没有 padding；left padding 只是 HF 静态 `[B,T]` 张量带来的产物。

Transformer 层。 Qwen3 和 Llama 3 的每一层都是 pre-norm 结构：先 `x + Attn(RMSNorm(x))`，再 `+ SwiGLU(RMSNorm(·))`。attention 用 GQA 加 RoPE；Qwen3 还在每个头的 Q、K 上各做一次 RMSNorm

(q_norm/k_norm)，位置在 RoPE 之前。下表是 Qwen3-8B 在 prefill 阶段（输入 `[B,T]`）的逐算子形状，S 表示已缓存长度加当前长度（modeling_qwen3.py @v5.19.0）：

步	算子	输出形状 / 要点
1	embed_tokens	[B,T,4096]
2	input_layernorm	在 fp32 中求均方根，形状不变
3	q/k/v_proj	Q [B,32,T,128]，K/V [B,8,T,128]
4	q_norm / k_norm	只在 head_dim 维做，在 RoPE 前
5	RoPE	只作用于 Q、K
6	cache.update	返回完整 K/V [B,8,S,128]
7	repeat_kv	扩成 [B,32,S,128]（组大小 4）
8	$QK^T/\sqrt{d}$ + mask + softmax	[B,32,T,S]，softmax 在 fp32
9	·V，o_proj，残差	[B,T,4096]
10	RMSNorm + SwiGLU	gate/up 12288，down 回 4096
11	36 层后 norm + lm_head	只取要采样的位置，[B,151936]

RoPE 只作用于 Q 和 K，而且 q·k 只依赖相对位置差，所以**缓存里存的是已经旋转过的 K**，decode 时只需对新 token 做一次旋转（推断，见 RoFormer arXiv 2104.09864）。

读形状时不要默认 `heads×head_dim = hidden`。例如 Qwen3-32B 的 q_proj 是 5120→8192 的扩张投影（Qwen3-32B config.json）。下表是本周会用到的模型参数（来自官方 config；KV/token 按 bf16 计算）：

模型	层数	Q/KV 头	hidden / FFN	KV/token
Qwen3-0.6B	28	16/8	1024 / 3072	112 KiB
Qwen3-8B	36	32/8	4096 / 12288	144 KiB
Qwen3-32B	64	64/8	5120 / 25600	256 KiB
Qwen3-30B-A3B	48	32/4	2048 / MoE 768×128	96 KiB
π0.5 Gemma-2B	18	8/1	2048 / 16384	18 KiB

Gemma-2B 一行来自 openpi 的 `gemma_2b` 配置（MQA，head_dim 256）（openpi gemma.py）。

关于 Qwen3-30B-A3B 的激活参数：按”attention + 8 个专家 + router + LM head”手算约为 3.04B，而模型卡写的是 3.3B。两者口径不同，官方口径「未核实」（Qwen3-30B-A3B README）。

当日阅读（约 3 小时）：

- 论文摘要与图：Transformer 1706.03762、GQA 2305.13245、RMSNorm 1910.07467、SwiGLU 2002.05202。
- 源码：通读 `modeling_qwen3.py @v5.19.0` 的 `Qwen3Attention` 和 `Qwen3DecoderLayer`。
- 背景：用 0.5 小时浏览 minbpe。

实验与代码

实验 1-1：环境与后台下载（约 1 小时）。运行上一节的 `nvidia-smi`、`topo -m`、`docker pull` 和 `hf download` 命令，把驱动版本和拓扑矩阵记进实验日志。下载放到后台，同时继续做后面的实验。

实验 1-2：字节级 BPE 与流式反分词（CPU，约 1 小时）。

```
cd ../labs
python lab01_bpe_toy.py --test           # 本机 8/8 PASS
python lab01_bpe_toy.py --merges 100     # 少学合并，观察中文被切得更碎
```

lab01 的关键代码是流式反分词器：它从缓冲区尾部往前找最后一个 UTF-8 首字节；如果这个字符还没收全，就先扣住这几个字节不输出（摘自../labs/lab01_bpe_toy.py）：

```python
class StreamDetokenizer:
    """ 逐 token 输入，只输出完整的 UTF-8 字符，不完整的尾部字节先暂存。"""

    def __init__(self, vocab: dict[int, bytes]) -> None:
        self.vocab, self.buf = vocab, b""

    def push(self, tid: int) -> str:
        self.buf += self.vocab[tid]
        cut = len(self.buf)
        for back in range(1, min(4, len(self.buf)) + 1):  # 从尾部找最后一个首字节
            need = utf8_len(self.buf[-back])
            if need:
                if need > back:          # 该字符还没收全：扣住这 back 个字节
                    cut -= back
                break
        out, self.buf = self.buf[:cut], self.buf[cut:]
        return out.decode("utf-8", errors="replace")
```

把它和两个工业实现对照看：

- vLLM 慢路径的做法是：解码结果以“`&#xfffd;`”结尾时返回空串。
- HF `TextStreamer` 的做法是：输出只截到最后一个空格；最后一个字符是 CJK 时立即输出（streamers.py）。

实验 1-3：最小解码器与 HF 等价性（CPU 或 1 卡，约 1.5 小时）。lab02 用约 250 行实现了 Llama/Qwen3 结构；lab03 把 HF 权重按名字映射进 lab02，然后逐位比对 logits，也比对贪心生成结果。

```
python lab02_tiny_llama.py --device cuda --test       # 本机 4/4 PASS
python lab03_hf_equivalence.py --device cuda --test   # 需 transformers 5.x
# 仅 H800（Linux）上：真实小模型；本机未运行，需要下载权重
python lab03_hf_equivalence.py --device cuda --dtype bfloat16 \
    --pretrained Qwen/Qwen3-0.6B
```

lab02 的 attention 把 GQA、RoPE、拼接 KV 和带偏移的因果掩码写在了一起（摘自../labs/lab02_tiny_llama.py）：

```python
    def attention(self, x: Tensor, cos: Tensor, sin: Tensor,
                  past: tuple[Tensor, Tensor] | None) -> tuple[Tensor, tuple]:
        c, (B, T, _) = self.c, x.shape
        q = self.q_norm(self.wq(x).view(B, T, c.n_heads, c.head_dim))
        k = self.k_norm(self.wk(x).view(B, T, c.n_kv_heads, c.head_dim))
        v = self.wv(x).view(B, T, c.n_kv_heads, c.head_dim)
        q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)
        if past is not None:  # 把新 token 的 K/V 接在历史后面
            k, v = torch.cat([past[0], k], 1), torch.cat([past[1], v], 1)
        S, rep = k.shape[1], c.n_heads // c.n_kv_heads
        kk, vv = k.repeat_interleave(rep, 2), v.repeat_interleave(rep, 2)
        s = torch.einsum("bthd,bshd->bhts", q, kk) * c.head_dim ** -0.5
        # 因果掩码：第 t 个新 token 的绝对位置是 S-T+t，只能看见不晚于它的 key
        qpos = torch.arange(S - T, S, device=x.device)[:, None]
        kpos = torch.arange(S, device=x.device)[None, :]
        p = s.masked_fill(kpos > qpos, float("-inf")).float().softmax(-1)
        o = torch.einsum("bhts,bshd->bthd", p.to(q.dtype), vv).reshape(B, T, -1)
        return self.wo(o), (k, v)
```

lab03 的权重映射表相当于真实推理引擎 weight loader 的缩影（摘自../labs/lab03_hf_equivalence.py）：

```
RENAME = {"self_attn.q_proj": "wq", "self_attn.k_proj": "wk", "self_attn.v_proj": "wv",
          "self_attn.o_proj": "wo", "self_attn.q_norm": "q_norm",
          "self_attn.k_norm": "k_norm", "mlp.gate_proj": "w_gate",
          "mlp.up_proj": "w_up", "mlp.down_proj": "w_down",
          "input_layernorm": "attn_norm", "post_attention_layernorm": "mlp_norm"}
```

lab03 只支持 lab02 能表达的子集：默认 RoPE、无 bias、全部是全注意力层。Qwen3-0.6B 的真实 config 是否落在这个子集里，本机没有验证过。如果脚本以”不支持的配置”退出，把具体是哪一项不支持也记下来，这本身就是在学习模型配置。

实验 1-4：真实 tokenizer 观察（CPU，约 1.5 小时，在 transformers 的 venv 中做）。

1. 加载 `Qwen/Qwen3-0.6B` 的 tokenizer，用 `apply_chat_template(messages, tokenize=False, add_generation_prompt=True)` 打印渲染后的字符串。
2. 对比传入 `enable_thinking=False` 前后，字符串末尾是否多了空的 think 块。
3. 对“Hello world”和“龘”做 `encode`，再用 `convert_ids_to_tokens` 逐个打印 token。
4. 把 lab01 的 `StreamDetokenizer` 改写成基于真实 tokenizer 的版本，与 `TextStreamer` 的输出对照。

OxyGen 对照：π0.5 的 prompt 每帧都在变

openpi 的 π0.5 预处理流程如下：

- 图像：缩放到 224×224（resize_with_pad），归一化到 [-1, 1]。
- 状态：用 `np.digitize` 离散化成 256 个 bin。
- prompt：拼成`"Task: …, State: …;\nAction: "`，用 PaliGemma 的 SentencePiece 分词，`max_token_len=200`（openpi tokenizer.py、pi0_config.py）。

也就是说，**状态被当作文本 token 写进了 prompt**，所以哪怕图像完全不变，每一帧的 token 序列也不同。π0.5 默认每帧的前缀约为 3×256+200 = 968 个 token（推断）。

这和 LLM 的 tokenize 层有两点区别：tokenizer 之前多了一步数值离散化；padding 用的是右侧 pad 加 mask。

验收标准与资源

验收标准：

- [概念] 不看资料写出上面 11 步的形状，并解释为什么缓存的是旋转后的 K、为什么 Qwen3-32B 的 q_proj 是 5120→8192。
- [概念] 手算 Qwen3-8B 的 KV/token：2×36×8×128×2 = **147,456 B**。
- [实验] 在 H800 服务器（Linux）上，lab01 8/8、lab02 4/4、lab03 4/4 全部 PASS，并记录 torch 版本和 CUDA 版本。
- [实验] 真实 tokenizer 上，“Hello world”应得到 `[9707, 1879]`，“龘”应得到 `[82912, 246]`。这两个预期值来自笔记作者的纯 Python 复现；如果不一致，记录差异。
- [实验] 自写的 streamer 对含生僻字的中文从不输出 U+FFFD，拼接后的结果与一次性 decode 逐字节一致。
- [实验] `lab03 --pretrained Qwen/Qwen3-0.6B` 要么 PASS，要么给出明确的不支持项，并附上解释。

资源：CPU 为主，加 1 张 GPU 约 1–2 GPU·h。后台下载约 165 GB（0.6B、8B、32B、30B-A3B、VL-7B）。

Day 2：prefill 吃算力，decode 吃带宽，KV cache 吃显存

目标：用 roofline 定量解释为什么 prefill 是 compute-bound、decode 是 memory-bound；在 H800 上测出 decode 的实际带宽利用率；核对 vLLM 是怎样把剩余显存换算成 KV 容量的；对 OxyGen 的 prefill、去噪和解码做分段计时。

理论：一条 ridge point 决定了所有优化的方向

Pope 等人的结论是推理算术的起点（Efficiently Scaling Transformer Inference, arXiv 2211.05102）：

- 每处理一个 token，前向约需 **2N FLOPs**（N 为参数量）。
- 无论 prefill 还是 decode，每次前向都要把权重和 KV 从 HBM 读一遍。
- 小 batch、短序列时，读权重的时间占主导；大 batch、长序列时，读 KV 的时间占主导。

Ridge point 与算术强度。用上文的分级数字计算：

- ridge point = 峰值算力 ÷ 带宽。BF16 为 989e12/3.35e12 ≈ **295 FLOP/B**，FP8 约 **591 FLOP/B**。
- 线性层（输入 [B,K] 乘权重 [K,N]）的算术强度约等于 B。
  - prefill 时，B 是批内全部 prompt token 数，通常上千，远超 295，所以受算力限制。
  - decode 时，B 是并发序列数，要到约 300 才碰到 ridge。
- **decode attention 的强度约等于 Hq/Hkv**，Qwen3-8B 为 4，与 batch 无关，所以永远受带宽限制。
- MLA 是例外：128 个 Q 头共享一个 576 维的 latent KV，强度约 242，接近 ridge。这就是 FlashMLA 能出现 compute-bound 配置的原因（推断，依据 FlashMLA @ba89a34 的数据）。

算例：Qwen3-8B，BF16，单卡，只计带宽下限。

- **batch=1**：每步约读 15.1 GB 权重（decode 只读 embedding 表中当前 token 那一行，所以总权重 16.38 GB 要减去 1.245 GB 的 embedding 表），得到 **4.5 ms/步，约 220 tok/s。**
- **batch=64、平均上下文 2048**：KV 读取量约 19.3 GB，每步合计约 34.4 GB，约 10.3 ms，聚合吞吐上限约 6,200 tok/s。
- 同一步的 GEMM 计算只需约 1.06 ms，所以仍然受带宽限制。

结论是：**加大 batch 能摊薄权重的读取，但摊不薄 KV 的读取**（推断，按上文参数计算）。

attention 的 FLOPs 在长上下文下也不能忽略：

- Qwen3-8B 在 4K 上下文时，每 token 的 attention 约 2.4 GFLOPs，约为 2N 的 16%。
- 32K 上下文时约 19 GFLOPs，已经超过 2N（推断）。

KV cache 公式与压缩。每 token 的 KV 字节数 = 2 × 层数 × KV 头数 × head_dim × 每元素字节数。

- GQA 把 KV 缩小 Hq/Hkv 倍。
- MQA 进一步缩到原来的 1/Hq。
- MLA 改为缓存低秩潜向量。DeepSeek-V2 摘要称，相对 DeepSeek 67B，KV cache 减少 93.3% (arXiv 2405.04434)。

DeepSeek 给出的每 token KV 对比是：V3 (MLA) 70.272 KB，Qwen-2.5 72B (GQA) 327.680 KB，LLaMA-3.1 405B 516.096 KB (arXiv 2505.09343)。可以用上面的公式交叉验证：Qwen2.5-72B 为 2×80×8×128×2 = 327,680 B，与论文一致。

HF 的 cache 实现。transformers v5.19 的 Cache 是一个按层的 Layer 对象列表 (cache_utils.py @v5.19.0)：

- `DynamicLayer` 用 `torch.cat` 让 KV 增长。
- `StaticLayer` 预分配固定长度，用 `index_copy_` 原地写入。

文档的原话是：DynamicCache “prevents you from taking advantage of most just-in-time (JIT) optimizations because the cache size isn’t fixed”；`cache_implementation="static"` 会自动编译 decode 阶段（kv_cache.md）。

原因在于：torch.compile 按输入形状特化，CUDA graph 捕获的是固定的指针和形状。静态 cache 把”写到第几个位置”从形状变成了张量里的数据，于是可以编译、可以 replay。vLLM/SGLang 的”paged KV + 按 batch size 分桶的 CUDA graph”是同一思路的工业化版本。

vLLM 的显存账。 KV 可用量 = 总显存 × `gpu_memory_utilization`（v0.31.0 默认 **0.92**）− profile run 测得的非 KV 内存 − CUDA graph 估计值。CUDA graph 的估计从 v0.21.0 起默认计入，可用 `VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=0` 关掉（gpu_worker.py @v0.31.0、cache.py @v0.31.0）。

按假设值估算 Qwen3-8B 在单卡 H800 上的情况（推断）：

- requested ≈ 79.6 GiB × 0.92 ≈ 73.2 GiB。
- 扣除权重 15.26 GiB、激活约 2 GiB、非 torch 内存约 1 GiB、CUDA graph 约 1 GiB，剩下约 54 GiB。
- 54 GiB ÷ 144 KiB/token ≈ **39.3 万 token**。按每请求 32K token 算，约 12 路并发；按 8K 算约 48 路。
- 换成 FP8 KV 后容量约翻倍。

当日阅读：

- Pope 论文第 2–3 节；DeepSeek-V2 的 Table 1。
- HF 文档 kv_cache.md 和 cache_explanation.md。
- gpt-fast：单 batch decode 的极致优化，即静态 KV 加 compile。
- CS336 的 L5 “GPUs, TPUs”和 L10 “Inference”（cs336.stanford.edu）。

实验与代码

实验 2-1：容量与 roofline 计算器（CPU，30 分钟）。下面命令中的硬件数字属于 T2/T3 来源。lab08 脚本本身不内置任何真实型号的参数。

```
python lab08_capacity_calculator.py --test            # 本机 13/13 PASS
python lab08_capacity_calculator.py ./cfg/qwen3-8b/config.json \
    --hbm-gb 80 --hbm-tbps 3.35 --tflops 989 --ctx 4096 --prompt 2048
```

lab08 的核心是这两个函数：先算出每 token 的 KV 和最大 KV 容量，再对每个 batch 取”算力时间”和”访存时间”中较慢的那个（摘自../labs/lab08_capacity_calculator.py）：

```
def analyze(m: dict, hw: dict) -> dict[str, float]:
    tp, wb, kb = hw["tp"], hw["weight_bytes"], hw["kv_bytes"]
    kv_tok = 2 * m["L"] * m["KV"] * m["D"] * kb   # 每 token 的 KV 字节（全模型）
    kv_heads_gpu = max(1, m["KV"] // tp)                      # tp > KV 时 KV 头被复制
    kv_tok_gpu = 2 * m["L"] * kv_heads_gpu * m["D"] * kb
    w_gpu = m["total"] * wb / tp
    budget = hw["hbm_gb"] * 1e9 * hw["mem_frac"] - w_gpu - hw["reserve_gb"] * 1e9
    return {"params": m["total"], "weight_bytes": m["total"] * wb,
            "kv_per_token": kv_tok, "kv_per_token_per_gpu": kv_tok_gpu,
            "max_kv_tokens": max(0, math.floor(budget / kv_tok_gpu)),
            "ridge": hw["tflops"] / hw["hbm_tbps"]}           # FLOP/字节


def step(m: dict, hw: dict, a: dict, B: int, S: int, prefill: bool) -> dict[str, float]:
    """prefill: 1 条序列 S 个 token; decode: B 条序列各 1 个 token, 上下文长 S。"""
    T = S if prefill else B
    attn = (2 * S * S if prefill else 4 * S * B) * m["L"] * m["H"] * m["D"]
    flops = 2 * m["matmul_active"] * T + attn
    wbytes = a["weight_bytes"]
    if m["E"] and not prefill:  # MoE 解码：只读被路由到的专家（均匀路由的上界）
        frac = min(1.0, B * m["k"] / m["E"])
        wbytes *= 1 - m["expert_frac"] * (1 - frac)
    kv = a["kv_per_token"] * (S if prefill else B * S)    # prefill 写 KV, decode 读 KV
    byts = wbytes + kv
    t = max(flops / (hw["tflops"] * 1e12 * hw["tp"]),           # 屋顶线：取两者较慢者
            byts / (hw["hbm_tbps"] * 1e12 * hw["tp"]))
    return {"flops": flops, "bytes": byts, "ai": flops / byts, "time": t,
            "tok_s": T / t,
            "bound": "compute" if flops / byts >= a["ridge"] else "memory"}
```

按脚本公式手算，Qwen3-8B 的参数量应为 **8,190,735,360**（脚本没有实际跑过真实 config），与 HF 的计数一致（Qwen3-8B config.json）。

batch=1、ctx=4096 时，脚本给出的下限约为 5.1 ms，比正文的 4.5 ms 慢。原因有两个：脚本把整张 embedding 表也算进了每步的读取量，而且加上了 4K 上下文的 KV。把这个差异讲清楚，正好是一道自检题。

实验 2-2：手写 prefill/decode 循环（GPU2，约 2 小时，在 transformers venv 中做）。下面的骨架只用了笔记中核实过的 API，**没有在任何机器上运行过**：

```
import time, torch
from transformers import AutoModelForCausalLM, AutoTokenizer, DynamicCache
name = "Qwen/Qwen3-8B"
tok = AutoTokenizer.from_pretrained(name, padding_side="left")
model = AutoModelForCausalLM.from_pretrained(name, dtype=torch.bfloat16).cuda()
msgs = [{"role": "user", "content": " 解释 KV cache"}]
inputs = tok.apply_chat_template(msgs, add_generation_prompt=True,
                                 return_tensors="pt", return_dict=True).to("cuda")
cache = DynamicCache(config=model.config)
torch.cuda.synchronize(); t0 = time.perf_counter()
out = model(**inputs, past_key_values=cache, use_cache=True)      # prefill
torch.cuda.synchronize(); print("prefill s", time.perf_counter() - t0)
nxt = out.logits[:, -1].argmax(-1, keepdim=True)
for _ in range(64):                                                # decode
    out = model(input_ids=nxt, past_key_values=cache, use_cache=True)
    nxt = out.logits[:, -1].argmax(-1, keepdim=True)
print(cache.get_seq_length(), torch.cuda.memory_allocated() / 2**30)
```

在这个骨架上做三组扫描和两组对照：

- 扫描 prompt 长度 T∈{128, 512, 2048}，记录 prefill 时间。
- 扫描 batch B∈{1, 8, 32, 128}，记录每步 decode 时间（多条 prompt 做 left padding）。
- 记录 KV 显存的增量。
- 对照组 A：不用 cache，每步把整段序列重新喂进去。
- 对照组 B：`generate(..., cache_implementation="static")`。

实验 2-3：vLLM 的 decode roofline（GPU0，约 2 小时）。

```
for b in 1 16 64 256; do for L in 512 2048; do
  CUDA_VISIBLE_DEVICES=0 vllm bench latency --model Qwen/Qwen3-8B \
    --num-iters-warmup 5 --num-iters 10 --batch-size $b \
    --input-len $L --output-len 128 --output-json day2_lat_b${b}_L${L}.json
done; done
```

实验 2-4：显存预算（GPU1，约 1 小时）。分别用 0.80 和 0.92 两个利用率各启动一次，再加上关闭 CUDA graph 估计的对照。从启动日志里 grep 出“Available KV cache memory”、“GPU KV cache size”和“Maximum concurrency”这三行（日志格式见 parallelism_scaling.md @v0.31.0）。

```
CUDA_VISIBLE_DEVICES=1 vllm serve Qwen/Qwen3-8B --port 8001 \
    --gpu-memory-utilization 0.80 2>&1 | tee day2_mem080.log
CUDA_VISIBLE_DEVICES=1 VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=0 \
    vllm serve Qwen/Qwen3-8B --port 8001 2>&1 | tee day2_mem092_nograph.log
```

实验 2-5：用 lab02/lab09 看小模型的 prefill/decode（任一 GPU，30 分钟）。lab02 的 `forward` 用 cache 的长度给新 token 定位，`prefill` 和 `decode_step` 只是同一个前向的两种调用方式（摘自../labs/lab02_tiny_llama.py）：

```
    @torch.no_grad()
    def forward(self, tokens: Tensor, cache: KVCache | None = None) -> Tensor:
        """tokens: [B, T] → logits [B, T, vocab]。cache=None 即无缓存整段前向。"""
        start = cache.seq_len() if cache is not None else 0
        cos = self.cos[start:start + tokens.shape[1]]
        sin = self.sin[start:start + tokens.shape[1]]
        x, new = self.tok_emb(tokens), []
        for i, blk in enumerate(self.layers):
            past = cache.layers[i] if cache is not None and cache.layers else None
            x, kv = blk(x, cos, sin, past)
            new.append(kv)
```

```
        if cache is not None:
            cache.layers = new
        return self.lm_head(self.norm(x))

    def prefill(self, tokens: Tensor) -> tuple[Tensor, KVCache]:
        cache = KVCache()
        return self.forward(tokens, cache), cache

    def decode_step(self, token: Tensor, cache: KVCache) -> Tensor:
        """token: [B, 1]; 原地扩展 cache, 返回 [B, 1, vocab]。"""
        return self.forward(token, cache)
```

运行方式：

```
python lab02_tiny_llama.py --device cuda --bench-prompt 2048 --bench-steps 64
python lab09_profile.py --device cuda --dim 2048 --layers 16 --test
```

在 H800（Linux，CUPTI 正常）上，lab09 应该能抓到 GPU kernel 事件。本机由于 CUPTI 不可用，这一项是 SKIP。

OxyGen 对照：前缀 KV 很小，瓶颈在 prefill 计算和权重带宽

π0/π0.5 的 Gemma-2B 是 MQA（只有 1 个 KV 头，head_dim 256），每 token 的 KV 只有 2×18×1×256×2 = **18,432 B**。π0.5 默认 968 个前缀 token 约占 17 MiB；OxyGen 在 N=30 时为每个请求预留 868+30 个槽位，约 15.8 MiB（推断）。

所以 KV 显存不是 OxyGen 的瓶颈。论文 Table 3 里 batch 4 时峰值显存只增加了约 15%，与此吻合。真正的瓶颈有两个：

- **prefill 计算**：在 RTX 4090 上，一次 prefill 约 44.0 ms，是 S=10 去噪（20.4 ms）的两倍多。
- **解码时的权重带宽**：每解码一步都要读一遍 2B 权重。batched decoding 把这份读取摊到多个语言请求上，所以每请求能耗最多下降 78%（OxyGen v3）。

这正是 Day 2 roofline 在 VLA 场景中的体现。

实验 2-6（GPU3，约 2 小时）：OxyGen 的 E1 和 E2。在你已复现的环境中把代码固定到 commit `1e88906`，用随机初始化做时延测试，不需要下载 checkpoint（Experiments.md）。

- **E1：分段计时。**在 `policy.py::infer_text_actions_continuous_batch` 中，`_prefill` 之后对输出调用 `jax.block_until_ready`，再分别计时 `_sample_actions_with_kv` 和 `_generate_n_tokens`。注意：现有的 `prefill_actions_ms` 是把 prefill 和去噪合在一起计时的。
- **E2：KV 大小核对。**在 `prefill` 返回后打印 kv_cache 各张量的 shape 和 nbytes，再改变 `max_token_len` 和图像的数量。注意要真正删掉对应的图像 key；只把 `image_mask` 改成 False 不会减少 token 数。

OxyGen 的 JAX 后端是在 4090、驱动 580.126.09、CUDA 13.0 上测试的；在 H800 上的兼容性「未核实」。

验收标准与资源

验收标准：

- [概念] 不看资料推出 295 FLOP/B、4.5 ms/token 和 6,200 tok/s 这三个数，并说清它们各自用了哪些假设。
- [实验] lab08 对 Qwen3-8B 输出 KV/token = 144.0 KiB，参数量 8.191B。
- [实验] batch=1 时 `bench latency` 得到的 tok/s 在理论上限（约 220）的 60–90% 之间。画出 batch 从 1 到 256 的每步时延曲线，并用”摊薄权重、摊不薄 KV”解释拐点。
- [实验] HF 手写循环中，KV 显存的实测增量与 144 KiB × token 数 × B 相差不超过 10%；不用 cache 时，每步耗时随长度线性增长。
- [实验] vLLM 日志中的 KV token 数，与”可用 KV 字节 ÷ 147,456”相差不超过 5%；利用率从 0.80 提到 0.92 后，可用 KV 增加约 9.5 GiB。

- [实验] OxyGen E1 各分段之和与 `frame_ms` 相差不超过 5%，100 次运行的变异系数小于 5%；E2 实测字节数与"token 数 × 18,432 × batch"相差不超过 1%。

资源：4 张卡分头跑，合计约 7 GPU·h。需要 Qwen3-8B。openpi 的快速测试首次运行时会把 `pi05_base` 下载到 `~/.cache/openpi`，大小「未核实」；随机初始化的实验不需要这一步。

Day 3：logits 到 token 之间还有七道工序

目标：理解 HF 和 vLLM 各自的 logits 处理顺序，以及顺序不同带来的后果；理解 logprobs 的口径；搞清楚停止条件分别在哪一层判断；理解结构化输出的 mask 机制和开销；复现"temperature=0 也不确定"，并用 batch invariance 消除它。

理论：顺序、口径与非确定性

处理顺序。 HF 在 `_get_logits_processor` 中先跑各类 penalty 和约束 processor，再跑用户自定义的 processor；只有 `do_sample=True` 时，才依次执行 Temperature、TopK、TopP、MinP（注释说明 min-p 必须放在 temperature 之后）（generation/utils.py）。vLLM 0.31.0 的顺序是（sampler.py @v0.31.0）：

1. 如果请求了 logprobs，先在原始 logits 上计算。
2. 转成 fp32。
3. 白名单、bad words。
4. min_tokens 和 logit_bias。
5. repetition、frequency、presence 三种 penalty。
6. temperature（小于 1e-5 视为 greedy）。
7. min_p。
8. top-k / top-p，然后采样。

两者都把 min-p 放在 temperature 之后，但 vLLM 把 min-p 放在 top-k/top-p 之前，HF 放在之后。在极端参数设置下，两边保留的 token 集合可能不同（推断）。

手算例子。logits 为 [2.0, 1.0, 0.5, −1.0]，T=0.5 时 softmax ≈ [0.842, 0.114, 0.042, 0.002]。再做 top-p=0.9：累积到第二个 token 时达到 0.956，所以保留前两个，重新归一化为 [0.881, 0.119]。

vLLM 的默认值与采样实现。 `SamplingParams` 的默认值是 `temperature=1.0, top_p=1.0, top_k=0, max_tokens=16`。注意 max_tokens 默认只有 16，这是一个常见的坑（sampling_params.py）。penalty 遵循 OpenAI API 的定义。随机采样用 `q.exponential_()` 再取 `argmax(probs/q)`：它与 Gumbel-max 等价，选中 i 的概率恰好是 p_i，而且避免了 multinomial 带来的 CPU-GPU 同步。

停止条件分三层：

- **token 级**：EOS、`stop_token_ids`、`max_tokens`，由 engine/scheduler 判断。
- **字符串级**：`stop`，由 detokenizer 判断。它必须缓冲尾部字符，以免把 stop 串的前缀先发给了用户。
- **`min_tokens`**：在达到最小长度之前屏蔽 stop token。

结构化输出。每一步都要算一个与词表等大的 bitmask，把非法 token 的 logit 置为 −inf。三个后端的做法不同：

- Outlines 把 regex/JSON 编译成 FSM，并预建索引（arXiv 2307.09702）。
- XGrammar 把 token 分为上下文无关（可以预先检查）和上下文相关两类，用 persistent stack 处理后者，并让 mask 计算与 GPU 前向重叠。论文称端到端"near-zero overhead"（arXiv 2411.15100）。
- llguidance 在 128k 词表上，单核 CPU 约 50µs 算出一次 mask（llguidance README）。

默认后端：

- vLLM 为 `auto`：先试 xgrammar，不支持时回退 guidance，再必要时回退 outlines。官方注释是"behavior is subject to change in each release"（structured_outputs.md @v0.31.0）。旧的 `guided_json` 字段已在 v0.12.0 移除。
- SGLang 默认 xgrammar（SGLang structured_outputs.mdx）。

非确定性。 Thinking Machines 用 Qwen3-235B 在 temperature 0 下跑了 1000 次，得到 **80 种不同输出。**他们的结论是：主因不是”并发加浮点”，而是 kernel 缺乏 batch invariance，即一个请求的数值结果依赖于同批的其他请求（Defeating Nondeterminism in LLM Inference）。

vLLM 提供 `VLLM_BATCH_INVARIANT=1`，目前为 beta，要求 compute capability ≥ 8.0(batch_invariance.md)。文档同时说明：即便打开它，也只在相同硬件、相同版本下可复现（reproducibility.md）。

当日阅读：

- 采样论文：Holtzman 1904.09751（top-p）、min-p 2407.01082。
- 上面的 Thinking Machines 博客全文。
- vLLM 源码：`sampler.py` 和 `topk_topp_sampler.py`。

实验与代码

实验 3-1：采样器（CPU，1 小时）。

```
python lab04_sampling.py --device cpu --test     # 本机 16/16 PASS
```

lab04 的 top-p、min-p 和”指数竞赛”采样（摘自../labs/lab04_sampling.py）：

```
def top_p(logits: Tensor, p: float) -> Tensor:
    """ 按概率从大到小累加，保留累计刚达到 p 的最小前缀（至少 1 个）。"""
    if p >= 1.0:
        return logits
    srt, idx = logits.sort(dim=-1, descending=True)
    probs = srt.softmax(-1)
    rm = (probs.cumsum(-1) - probs) >= p      # 排在前面的概率和已够 p → 它多余
    rm[..., 0] = False
    return logits.masked_fill(torch.zeros_like(rm).scatter(-1, idx, rm), NEG_INF)


def min_p(logits: Tensor, mp: float) -> Tensor:
    """ 保留 prob >= mp × 最大概率 的 token。"""
    if mp <= 0:
        return logits
    probs = logits.softmax(-1)
    keep = probs >= mp * probs.max(-1, keepdim=True).values
    return logits.masked_fill(~keep, NEG_INF)


def sample(logits: Tensor, gen: torch.Generator | None = None) -> Tensor:
    """ 按 softmax(logits) 抽样：argmax(p_i / E_i)，E_i ~ Exp(1) 独立。
    E_i/p_i ~ Exp(速率 p_i)，最小者是 j 的概率恰为 p_j / Σp（”指数竞赛”）。
    这样无需 torch.multinomial，推理引擎中常见这种写法。"""
    probs = logits.softmax(-1)
    e = torch.empty_like(probs).exponential_(1.0, generator=gen)
    return (probs / e).argmax(-1)
```

练习：lab04 的 process 采用 HF 顺序（top-k → top-p → min-p）。把它改成 vLLM 的顺序（min-p → top-k/top-p），找出一组 logits，使两种顺序保留的集合不同。

实验 3-2：batch 依赖的非确定性（GPU0 和 GPU1 并行，约 2 小时）。下面的 API 写法取自笔记中的 vLLM 文档，未运行过：

```
from vllm import LLM, SamplingParams
llm = LLM(model="Qwen/Qwen3-8B", tensor_parallel_size=1)
sp = SamplingParams(temperature=0, max_tokens=256, logprobs=5)
alone = llm.generate([target_prompt], sp)
mixed = llm.generate([target_prompt] + other_63_prompts, sp)
```

步骤：

1. 比较 `alone` 和 `mixed` 中目标 prompt 的 token 序列，找出首个分歧位置，记录该位置的 logprob 差。输出对象的字段名以当版文档为准。
2. 在 GPU1 上设置 `VLLM_BATCH_INVARIANT=1` 后重做一遍。
3. 换成 `temperature=0.7，seed=42`，测试 min_p 与 top_p 的组合，用 lab04 手算对照。

实验 3-3：结构化输出的开销（GPU2 和 GPU3 并行，约 1.5 小时）。

```
CUDA_VISIBLE_DEVICES=2 vllm serve Qwen/Qwen3-8B --port 8002 \
    --structured-outputs-config.backend xgrammar
CUDA_VISIBLE_DEVICES=3 vllm serve Qwen/Qwen3-8B --port 8003 \
    --structured-outputs-config.backend guidance
```

写一个小的异步 OpenAI 客户端：

- 在请求的 `extra_body={"structured_outputs": {"json": schema}}` 中传入一个嵌套的 JSON schema。
- 并发度分别取 1、32、128，与无约束请求对比 TPOT 和吞吐。
- 记录首个请求的 grammar 编译延迟。
- 对每条输出做 `json.loads` 和 schema 校验。

实验 3-4：真实 tokenizer 的流式输出（CPU，30 分钟）。用 Day 1 的自写 streamer 和 `TextStreamer` 处理同一段含生僻字的中文，逐 token 对比两者的输出时机。

OxyGen 对照：每个语言请求都带着自己的随机状态跨帧

OxyGen 的 `IncrementalTextState` 被注册为 JAX pytree，它的数据字段包括 `rng，last_logits，output_tokens，kv_cache，current_step，is_finished`（pi05.py @1e88906）。也就是说，每个语言请求的采样随机状态和上一步的 logits，都随 KV 一起跨帧保存和恢复。这与 vLLM 给带 seed 的请求单独维护一个 `torch.Generator` 是同一回事。

还有一个工程上的教训：Experiments.md 记录了在 Thor 上，编译后的单 token 解码曾产生非法的采样概率，所以稳定模式设置 `OPENPI_TORCH_COMPILE_TEXT_DECODE=0`。编译带来的数值问题，最先暴露在采样器的边界上。

由此还能推出一点：OxyGen 每帧的 batch 组成都在变，而 batch invariance 的结论告诉我们，即使贪心解码，语言输出也可能与单独执行时不同（推断，待你用 E3 的配置验证）。

验收标准与资源

验收标准：

- [概念] 手算出上面的 [0.881, 0.119]；能解释 `argmax(p/q)` 为什么等价于按 p 采样；能说出 vLLM 默认返回的 logprobs 是**处理前**的。
- [实验] lab04 16/16 PASS，并给出一组 HF 顺序与 vLLM 顺序保留集合不同的反例。
- [实验] 给出默认模式下的分歧率、首个分歧位置，以及开启 batch invariance 后的一致性结果和吞吐比。
- [实验] 两个后端 × 三档并发的 TPOT 差异表；JSON 合法率 100%；首请求编译延迟。
- [实验] 流式输出从不出现 U+FFFD。

资源：4 卡分头跑，约 5 GPU·h，模型用 Qwen3-8B。

Day 4：调度器决定谁先算，KV 管理决定能装多少

目标：理解 continuous batching、PagedAttention、prefix caching、chunked prefill 和抢占这五个机制，以及它们之间的耦合关系；读懂 nano-vllm 的 scheduler 和 block manager，并加上自己的打印；在 vLLM 和 SGLang 上测出 token 预算和 prefix cache 对 TTFT、ITL 的影响。

理论：iteration-level 调度加分页 KV，就是现代引擎的骨架

从 static 到 iteration-level。早期的批处理有两种：

- static batching：凑齐一批后一起跑完，短请求要陪着长请求等。
- dynamic batching：按时间窗或批大小凑批，但粒度仍然是"请求"。

Orca 提出了 iteration-level scheduling：每生成一个 token，就重新决定一次 batch 成员，完成的请求立即退出，新请求立即加入。这就是 continuous batching（Orca OSDI'22；会议页面本身在笔记中「未核实」）。

vLLM V1 调度器（v0.31.0）。前面提到过，它用 `num_computed_tokens` 追赶 `num_tokens_with_spec` 这一个抽象统一了 prefill 和 decode。具体做法（scheduler.py @v0.31.0、config/scheduler.py @v0.31.0）：

- 每步有一个 token 预算。先排 RUNNING 请求，再排 WAITING 请求。
- 预算不够时，长 prompt 会被切块。
- `long_prefill_token_threshold` 用来限制单个长 prefill 能占多少预算。v0.31.0 起它会根据等待中的 prefill 数量自适应调整。

H800 属于"≥70 GiB 且不是 A100"这一档，对应的默认值是（arg_utils.py @v0.31.0）：

- `max_num_batched_tokens`：serve 为 **8192**，`LLM` 类为 **16384**。
- `max_num_seqs`：**1024**。

PagedAttention。 KV 按固定大小的 block 分配，每个请求用一张 block table 把逻辑块映射到物理块。这样做消除了外部碎片，并支持块级共享。论文称浪费"near-zero"，吞吐提升 2–4×（arXiv 2309.06180）。v0.31.0 的默认 block 大小为 16（cache.py @v0.31.0）。

抢占。KV 不够时，FCFS 策略选择最后加入的请求作为牺牲者：释放它的全部 block，`num_computed_tokens` 归零，放回等待队列的队首。这是 **recompute 式抢占**，没有 swap 路径。KV 紧张时的表现是 `vllm:num_preemptions` 上升、尾延迟变差，而不是出现 PCIe 流量。

Prefix caching，两个引擎做法不同：

- **vLLM APC**：只缓存**满块**，键为 `hash(parent_hash, block_tokens, extra_hashes)`。extra 中包括 LoRA ID、多模态 hash 和 `cache_salt`，默认哈希算法为 sha256。空闲块放在 free queue 里，按 LRU 从队首驱逐；请求结束时，块以**逆序**放回队尾，所以请求尾部的块最先被驱逐（prefix_caching.md @v0.31.0）。
- **SGLang**：用 RadixCache（`match_prefix / insert / evict`），默认开启，驱逐策略默认 lru。v0.5.21 起，prefix cache 默认运行在 Rust core 上（radix_cache.py @v0.5.21、SGLang v0.5.21 release）。SGLang 把 KV 不够时的撤回叫做 retract；如果请求频繁被 retract，帮助文字建议调大 `--schedule-conservativeness`。

当日阅读（按此顺序）：

1. Orca。
2. PagedAttention。
3. Sarathi-Serve 2403.02310。
4. SGLang 2312.07104。
5. nano-vllm（约 1,200 行），按 scheduler → block manager → model runner 的顺序读。
6. 回到 vLLM 读 `schedule()`、`block_pool.py` 和 `kv_cache_utils.py`。

实验与代码

实验 4-1：分页 KV 与前缀缓存（CPU，1 小时）。

```
python lab05_paged_kv.py --device cpu --test     # 本机 9/9 PASS
```

lab05 的块池把"空闲队列"和"LRU 缓存"合成了同一个结构：引用计数为 0 的缓存块留在空闲队列里，等到真正被重新分配时才驱逐（摘自../labs/lab05_paged_kv.py）：

```
    def allocate(self) -> int:
        if not self.free:
            raise MemoryError("KV 块耗尽")
        blk, _ = self.free.popitem(last=False)
        if blk in self.hash_of:                          # 仍缓存着旧前缀：此刻驱逐
            del self.cached[self.hash_of.pop(blk)]
            self.evictions += 1
        self.ref[blk] = 1
        return blk

    def lookup(self, h: str) -> int | None:
        blk = self.cached.get(h)
        if blk is not None:
            if self.ref[blk] == 0:
                del self.free[blk]                       # 命中空闲缓存块：从空闲队列“复活”
            self.ref[blk] += 1
            self.hits += 1
        return blk

    def release(self, blk: int) -> None:
        self.ref[blk] -= 1
        assert self.ref[blk] >= 0
        if self.ref[blk] == 0:
            self.free[blk] = None                        # 放队尾：最后被驱逐
```

```
    def free(self, sid: int) -> None:
        for blk in reversed(self.tables.pop(sid)):       # 逆序释放：尾部块先被驱逐
            self.pool.release(blk)
        del self.tokens[sid], self.hashes[sid]
```

实验 4-2：调度模拟（CPU，1 小时）。lab06 的成本模型是示意性的，只能看趋势，不能看绝对值。

```
python lab06_scheduler_sim.py --test                 # 本机 13/13 PASS
python lab06_scheduler_sim.py --kv-blocks 300        # 以下参数组合本机未单独运行
python lab06_scheduler_sim.py --budget 2048 --rate 8
```

lab06 中 chunked 策略的一步：先给 running 请求排工作，必要时抢占最后加入的请求；再用剩余预算切块接纳新请求（摘自../labs/lab06_scheduler_sim.py）：

```
    def schedule_running(self, budget: float) -> list[tuple[Req, int]]:
        sched = []
        for r in list(self.running):
            n = int(min(r.ntok - r.computed, budget))
            if r not in self.running or n <= 0:
                continue
            while not self.alloc(r, n):  # 显存不足：抢占最后加入的请求
                victim = self.running[-1]
                self.preempt(victim)
                if victim is r:
                    break
            if r in self.running:
                sched.append((r, n))
                budget -= n
        return sched
```

```
        else:
            sched = sim.schedule_running(p.budget)
            sim.admit_waiting(sched, p.budget - sum(n for _, n in sched), chunk=True)
```

本机默认负载下的结果：

- static 吞吐为 416 tok/s，TTFT p50 约 20 秒，说明它根本跟不上到达速率。
- 两种 continuous 策略的吞吐都是 738。chunked 的 TPOT p99 更低（25.2 vs 27.9），代价是 TTFT 略高。
- KV 收紧到 300 块后，chunked 的抢占次数为 839，prefill_first 只有 58。

把这些现象讲清楚，是今天的概念核心。

实验 4-3：nano-vllm（GPU0，约 2 小时）。

1. 用 `pip install git+https://github.com/GeeeekExplorer/nano-vllm.git` 安装（README 原文）。
2. 按 README 的示例运行 Qwen3-0.6B，具体脚本名以 README 为准（「未核实」）。
3. 在 scheduler 里每步打印三个数：prefill token 数、decode 序列数、空闲块数。把 `block_size` 改小一档，观察 prefix 命中的变化。

实验 4-4：并发扫描与 token 预算（GPU1，约 2 小时）。

```
# 终端 A（常驻）：默认 serve 预算 8192
CUDA_VISIBLE_DEVICES=1 vllm serve Qwen/Qwen3-8B --port 8001
# 终端 B：
for c in 1 8 32 128 512; do
  vllm bench serve --model Qwen/Qwen3-8B --base-url http://127.0.0.1:8001 \
    --dataset-name random --random-input-len 1024 --random-output-len 256 \
    --num-prompts $((c*10)) --max-concurrency $c --ignore-eos \
    --num-warmups 10 --percentile-metrics ttft,tpot,itl,e2el \
    --metric-percentiles 50,99 --save-result --result-dir ./day4_expA
done
curl -s http://127.0.0.1:8001/metrics | grep -E \
  "vllm:(num_requests_running|num_requests_waiting|kv_cache_usage_perc)"
```

然后分别用 `--max-num-batched-tokens 2048` 和 `--max-num-batched-tokens 16384` 重启服务，把负载换成 `--random-input-len 8192` 的长 prompt 再测一遍，比较 TTFT 和 ITL P99。

实验 4-5：两个引擎的 prefix cache（GPU2 跑 vLLM，GPU3 跑 SGLang，约 2 小时，两者用不同的容器或 venv）。

```
CUDA_VISIBLE_DEVICES=2 vllm serve Qwen/Qwen3-8B --port 8002
vllm bench serve --model Qwen/Qwen3-8B --base-url http://127.0.0.1:8002 \
  --dataset-name prefix_repetition --prefix-repetition-prefix-len 2048 \
  --prefix-repetition-suffix-len 128 --prefix-repetition-num-prefixes 8 \
  --prefix-repetition-output-len 128 --num-prompts 800 \
  --max-concurrency 64 --ignore-eos
# 对照组：重启时加 --no-enable-prefix-caching
#  （--no- 形式由 BooleanOptionalAction 推断，启动日志中确认 APC 已关）
CUDA_VISIBLE_DEVICES=3 python -m sglang.launch_server \
  --model-path Qwen/Qwen3-8B --port 30000 --enable-metrics
python -m sglang.benchmark.serving --backend sglang \
  --host 127.0.0.1 --port 30000 --dataset-name generated-shared-prefix \
  --gsp-num-groups 8 --gsp-prompts-per-group 100 \
  --gsp-system-prompt-len 2048 --gsp-question-len 128 \
  --gsp-output-len 128 --max-concurrency 64
# 对照组：启动参数加 --disable-radix-cache；记录 sglang:cache_hit_rate
```

`python -m sglang.bench_serving` 在 v0.5.21 中已经是弃用路径，会发出 FutureWarning，所以上面用的是 `sglang.benchmark.serving`（benchmark/serving.py @v0.5.21）。

OxyGen 对照：帧粒度的 continuous batching，加上静态预留的 KV

OxyGen 的 `ContinuousBatchManager` 用 `active_states: dict[request_id → IncrementalTextState]` 管理所有活跃请求（kv_cache_manager.py @1e88906）。每帧的流程是：

1. 新请求做一次 `_prefill(max_decoding_steps=...)`。
2. `_stack_incremental_states` 把所有状态拼成一个 batch。
3. 调用一次 `_generate_n_tokens(k)` 推进 k 步。
4. 拆分状态，未完成的写回 manager，已完成的移除。

用今天学的概念重新描述它：

- **iteration-level scheduling**：迭代粒度是”帧”，新请求只在帧边界加入。
- **KV 是静态预留，不分页**：每个请求预留 `prefill_size+N` 个槽位。有效前缀在 `[0, prefill_len)`，解码 token 从 `prefill_size` 开始写，中间是 padding，所以必须用 `_gather_last_valid_token` 取种子 embedding。
- **前缀共享发生在同一帧内、两个专家之间**，而不是像 vLLM 的 ref_cnt 那样在多个请求之间共享。
- **前缀不能跨帧复用**：前缀内部是双向注意力，状态又写进了文本，所以第 1 层之后所有前缀 token 的 KV 都随状态变化，LLM 式的精确 prefix caching 无法跨帧复用。VLA-Cache 走的是近似复用的路线 (arXiv 2502.02175)。

实验 4-6（GPU0 在 nano-vllm 之后，约 2 小时）：OxyGen 的 E3。下面的参数组合取自笔记中 Experiments.md 的参数清单，未在 H800 上运行过：

```
uv run python -m experiments.run_experiments \
  --settings baseline shared_kv continuous_batching \
  --policies pi05_o2_libero \
  --max-decoding-steps 10 20 30 --steps-per-frame 1 5 10 \
  --checkpoint-dir /tmp/oxygen_random_init_checkpoint \
  --results-dir experiments/results_h800_jax --gpu 0
```

`N % k ≠ 0` 的组合会被脚本跳过。按 Little 定律推断：在 uniform 到达（rate=1）下，每个请求存活 N/k 帧，所以稳态 batch ≈ N/k。

验收标准与资源

验收标准：

- [概念] 能解释为什么”只缓存满块”就不需要 copy-on-write；为什么要逆序释放；为什么紧 KV 下 chunked 的抢占远多于 prefill_first（它让更多请求同时处于运行状态，容量一紧就反复抢占和重算）。
- [实验] lab05 9/9、lab06 13/13 PASS。nano-vllm 打出每步的 batch 组成，并附一份带注释的调用链笔记。
- [实验] 画出”并发—吞吐—TPOT”曲线，用 `num_requests_waiting` 和 `kv_cache_usage_perc` 解释拐点；写出预算 2048 与 16384 下 TTFT 和 ITL P99 的取舍。
- [实验] 两个引擎各给出一张缓存开/关的表 (TTFT P50、吞吐、命中率)。命中率应接近 2048/(2048+128) ≈ 94%，低于这个值的部分用”首次出现时的 miss”来解释。另写一页对比 `block_pool.py` 与 `radix_cache.py` 的数据结构。
- [实验] OxyGen E3：`avg_batch_size` 与 N/k 偏差不超过 0.5；加速比随 N/k 增大的趋势与论文一致。

资源：约 9 GPU·h，模型为 Qwen3-0.6B 和 Qwen3-8B。

Day 5：引擎是一组进程，服务质量要用 SLO 来衡量

目标: 画出 vLLM 和 SGLang 的进程结构，并用 `ps` 验证；理解 CUDA graph、async scheduling 和 overlap 调度各自在消除什么开销；掌握 TTFT/ITL/TPOT/goodput 的定义和压测方法学；用一次 speculative decoding 和一次（可选的）PD 分离实验，理解”用额外计算换时延”和”用分离换隔离”这两种思路。

理论：三类开销，三种隐藏手段

为什么需要 CUDA graph。对 8B 级模型，decode 一步的 GPU 时间只有几毫秒，但每层约 10 个 kernel，几十层下来就是数百次 launch。不用 CUDA graph 时，CPU 发射和 Python 调度的开销会在 timeline 上表现为 GPU 空泡（推断，依据 vLLM profiling.md @v0.31.0 中的 kernel 构成）。

vLLM v0.31.0 的相关默认值（vllm/config/vllm.py @v0.31.0）：

- `optimization_level` 默认 O2，即 torch.compile 加 `FULL_AND_PIECEWISE`。
- `enforce_eager=True` 会同时关闭 compile 和 CUDA graph。
- async scheduling 解析后通常为 True，让 CPU 调度与 GPU 执行重叠。

SGLang 用 overlap event loop 达到类似的效果，`--disable-overlap-schedule` 默认为 False，即默认开启 overlap（schedule.py @v0.5.21）。

指标与方法学。先明确定义：

- TTFT：从请求发出到收到第一个 token。
- ITL：相邻两个 token 之间的间隔。
- TPOT：每个请求的 (E2E − TTFT) / (输出 token 数 − 1)。
- goodput：满足 SLO 的吞吐。`--goodput ttft:...tpot:...` 的定义引用了 DistServe（vllm/benchmarks/serve.py @v0.31.0）。

两种压测方式**不能混着比较**：

- `--request-rate` 取有限值时是 open-loop，到达服从 Poisson 过程，模拟真实流量。
- `--max-concurrency` 是 closed-loop，固定在途请求数。

两个引擎压测工具的默认值也不一样，比较前必须显式对齐（sglang benchmark/serving.py @v0.5.21）：

- vLLM bench：默认**不** ignore EOS，warmup 为 0，数据集为 random。
- SGLang bench：默认 ignore EOS，warmup 为 1，数据集为 sharegpt。

服务端指标方面，vLLM 在 `/metrics` 暴露以下几类（loggers.py @v0.31.0）：

- 延迟：`vllm:time_to_first_token_seconds`、`vllm:inter_token_latency_seconds`、`vllm:request_queue_time_seconds`。
- 负载与缓存：`vllm:kv_cache_usage_perc`、`vllm:num_preemptions`、`vllm:prefix_cache_hits`。
- 投机解码：`vllm:spec_decode_num_accepted_tokens`。

SGLang 必须加 `--enable-metrics` 才会暴露指标（production_metrics.mdx @v0.5.21）。

Speculative decoding。 Leviathan 和 Chen 的两篇论文都用 rejection sampling 保证最终输出的分布与目标模型完全一致（arXiv 2211.17192、arXiv 2302.01318）。之后的变体包括 Medusa、EAGLE-1/2/3 和 MTP（EAGLE-3 2503.01840）。两个引擎的支持情况：

- vLLM 0.31.0 的 `method` 可取 ngram、medusa、draft_model、suffix，以及 EAGLE 类（speculative.py @v0.31.0）。
- SGLang 0.5.21 内置 EAGLE、EAGLE3、NEXTN、NGRAM 等（spec.py @v0.5.21）。

它的本质是用额外的计算（草稿加验证）去摊薄”每步读一遍权重”的固定成本。在高并发、已经接近 compute-bound 时，收益会下降（推断）。

PD 分离。相关的三篇代表论文：

- DistServe 以 goodput 为目标拆分 prefill 和 decode（arXiv 2401.09670）。
- Splitwise（arXiv 2311.18677）。
- Mooncake 采用以 KVCache 为中心的分离架构（arXiv 2407.00079）。

工程实现上：

- vLLM 的 KV connector 在 `kv_transfer/kv_connector/v1/` 下，包括 nixl、lmcache、mooncake 和 offloading（源码树 @v0.31.0）。
- SGLang 的 PD 默认传输后端是 mooncake，也支持 NIXL（pd_disaggregation.mdx @v0.5.21）。

PD 分离消除了 prefill 对 decode 的打断，ITL 尾延迟更稳定，代价是 TTFT 多了 KV 传输时间，而且两侧负载不均衡时资源利用率反而下降。

其他引擎只需知道定位，它们的版本号在 PyPI 上核实过，定位描述大多「未核实」：

- TensorRT-LLM：稳定版 1.2.1。
- NVIDIA Dynamo：1.5.1，属于数据中心编排层。
- LMDeploy：0.18.0。
- Ray：2.59.0（含 Ray Serve LLM）。

当日阅读：

- 设计文档：arch_overview、model_runner_v2.md、cuda_graphs.md。
- 论文：DistServe、Leviathan。
- mini-sglang（约 5,000 行，仅支持 Linux）：读 overlap scheduling 和 chunked prefill。

实验与代码

实验 5-1：open-loop 速率扫描与 goodput（GPU0，约 2.5 小时）。

```
CUDA_VISIBLE_DEVICES=0 vllm serve Qwen/Qwen3-8B --port 8000
for r in 2 4 8 16 32; do
  vllm bench serve --backend openai --model Qwen/Qwen3-8B \
    --base-url http://127.0.0.1:8000 --dataset-name random \
    --random-input-len 1024 --random-output-len 256 \
    --num-prompts $((r*60)) --request-rate $r \
    --goodput ttft:500 tpot:50 --ignore-eos --num-warmups 10 \
    --save-result --result-dir ./day5_rate
done
```

实验 5-2：CUDA graph 的作用（GPU1，约 2 小时）。用 `vllm serve Qwen/Qwen3-8B --port 8001 --enforce-eager` 和默认配置各起一次服务，在并发 1 和 64 下用同一条 bench 命令比较 TPOT 和启动时间。v0.31.0 起 `--enforce-eager` 还会同时关闭 JIT kernel warmup（release v0.31.0）。

两种配置都用 torch profiler 采样，在 Perfetto 里比较 decode 步之间的空泡：

```
CUDA_VISIBLE_DEVICES=1 vllm serve Qwen/Qwen3-8B --port 8001 --profiler-config \
  '{"profiler": "torch", "torch_profiler_dir": "./vllm_profile"}'
# 压测时给 vllm bench serve 加 --profile 少量采样
```

`--profiler-config` 需要 vLLM ≥ v0.13.0。文档提醒不要在生产环境开启 profiling(profiling.md @v0.31.0)。

实验 5-3：SGLang 的 overlap 调度（GPU2，约 1.5 小时）。默认配置和加上 `--disable-overlap-schedule` 各起一次服务。两边都用 `vllm bench serve --backend openai` 打同一份负载，保证客户端一致。

实验 5-4：ngram 投机解码（GPU3，约 1.5 小时）。JSON 写法是按字段名组合出来的，属推断：

```
CUDA_VISIBLE_DEVICES=3 vllm serve Qwen/Qwen3-8B --port 8003 \
  --speculative-config \
  '{"method":"ngram","num_speculative_tokens":4,"prompt_lookup_max":4}'
```

用 `prefix_repetition` 数据集这类输入输出高度重复的负载压测，从 `/metrics` 读出 `vllm:spec_decode_num_draft_tokens` 和 `vllm:spec_decode_num_accepted_tokens`，算出接受率，并在并发 1 和 64 下与无投机解码对比 ITL。

实验 5-5：mini-sglang（任一 GPU 空闲时，约 1 小时）。运行 `python -m minisgl --model "Qwen/Qwen3-0.6B"`（也可以加 `--shell` 进入交互模式），对照阅读它的 overlap 调度实现。

实验 5-6（可选，取决于网卡）：SGLang PD 分离。命令来自文档原文，需要 RDMA 设备，并先安装 `mooncake-transfer-engine`。如果机器没有 RDMA，Mooncake 和 NIXL 能否走纯 NVLink 或 TCP「证据不足」，这个实验可能无法完成。

```
python -m sglang.launch_server --model-path Qwen/Qwen3-8B \
  --disaggregation-mode prefill --port 30000 \
  --disaggregation-ib-device mlx5_roce0
python -m sglang.launch_server --model-path Qwen/Qwen3-8B \
  --disaggregation-mode decode --port 30001 --base-gpu-id 1 \
```

```
  --disaggregation-ib-device mlx5_roce0
python -m sglang_router.launch_router --pd-disaggregation \
  --prefill http://127.0.0.1:30000 --decode http://127.0.0.1:30001 \
  --host 0.0.0.0 --port 8000
```

OxyGen 对照：动作截止期就是一种 per-iteration SLO

OxyGen v3 把问题定义为"非对称截止期下的多任务并行"：动作有逐帧截止期，语言只有软截止期，优化目标是在满足动作截止期的前提下最大化语言吞吐（OxyGen v3）。用今天的语言说，这就是一个"带 per-iteration SLO 的 goodput 问题"。固定的 k 起的作用类似 chunked prefill 中"每迭代 token 预算"：都是给每一轮的工作量设上限，以保护时延敏感的那一路。

但要注意，论文定义的 action frequency = H/T 是**吞吐型指标**。真正的反应时延，是新请求从到达到动作就绪所需的时间。

另外，MPS 并行是空间共享。它在 4090 上收益有限，在 Thor 上反而更慢，原因是两个进程都会重复计算前缀。

实验 5-7（GPU3 在投机解码之后，约 2 小时）：OxyGen 的 E4 和 E5。

- **E4：比较三种后端。**分别以 `OPENPI_TORCH_COMPILE=0/1` 和 `OPENPI_TORCH_COMPILE_DENOISE_CUDAGRAPHS=0/1` 运行 PyTorch 版本（加 `--pytorch-device cuda:0`），与 JAX 对照，记录编译耗时和稳态的 prefill/denoise/decode 时延。参考值：realtime-vla 在 4090 上用 CUDA graph 等手段，把 π0 三视角的时延从 53.7 ms 降到 27.3 ms（arXiv 2510.26742）。
- **E5：截止期松弛度。**取 f_min=50 Hz、H=10，则每帧预算 T_max=200 ms。对每一帧计算 slack = T_max − frame_ms，报告 miss rate 和 p99 slack，并找出满足 miss rate < 1% 的最大 k。

验收标准与资源

验收标准：

- [概念] 画出 vLLM（API server / EngineCore / Worker）和 SGLang（TokenizerManager / Scheduler / Detokenizer）的进程图；说出 vLLM bench 与 SGLang bench 的三处默认差异；能解释投机解码为什么不改变输出分布。
- [实验] 画出 rate–goodput 曲线，标出在 TTFT 500 ms、TPOT 50 ms 的 SLO 下能承受的最大请求速率。
- [实验] 给出 eager 与默认配置在并发 1 下的 TPOT 比值，并附 Perfetto 截图说明空泡。
- [实验] 给出 overlap 开/关的对比表；投机解码的接受率，以及它对低并发和高并发 ITL 的不同影响。
- [实验] 所有压测都保存 `--save-result` 的原始 JSON 和启动命令。
- [实验] OxyGen E5 的 miss rate 表，以及满足条件的最大 k。

资源：约 10 GPU·h。PD 实验额外需要 2 卡加 RDMA，属于可选。

Day 6：H800 的 400 GB/s NVLink 改写了并行选型

目标：实测 4 卡之间的 all-reduce 带宽，并代回 TP 通信量公式；为 8B、32B 和 30B-A3B MoE 三类模型选出并行方案，用数据证明；用 nsys 拿到第一份多卡 timeline；理解 Hopper 上的量化和关键 kernel 选型。

理论：通信量公式、并行选型规则与量化路线

Megatron 式 TP 的通信。 MLP 的第一个 GEMM 按列切分，第二个按行切分；attention 按头切分，输出投影按行切分。所以每层前向**只需 2 次 all-reduce**（Megatron-LM arXiv 1909.08053）。

nccl-tests 的带宽口径：algbw = S/t；all-reduce 的 busbw = algbw × 2(n−1)/n。busbw 可以直接和硬件峰值比较（nccl-tests PERFORMANCE.md）。

两个通信算例（推断）：

- **Qwen3-8B、TP4、8192 token 的 prefill**：单次 all-reduce 的消息大小 S = 8192×4096×2 B = 64 MiB。假设 busbw 约 150 GB/s（要用今天的实验实测），每次约 0.67 ms，72 次合计约 48 ms。同一步每卡的计算量，按峰值算约 34 ms，按 60% MFU 算约 57 ms。也就是说，在 H800 上通信与计算处于同一量级。
- **decode、batch 64**：消息只有 512 KiB，单次耗时由 NCCL 的固定延迟决定。72 次加起来约 1 ms 量级，相对 4–10 ms 的单步时间不能忽略。

并行选型规则。 vLLM 官方的建议是：单卡放得下就不要做分布式；放不下时，单机内用 TP；没有 NVLink 时用 PP（parallelism_scaling.md @v0.31.0）。

对 MoE（data_parallel_deployment.md、expert_parallel_deployment.md）：

- attention 用 DP；加上 `--enable-expert-parallel` 后，专家层改为 EP，且 EP_SIZE = TP×DP。
- 各 DP rank 必须同步前向，空闲的 rank 也要执行 dummy forward。

结合 H800 的 NVLink 只有 H100 的 4/9，在 4×H800 SXM 上的推荐是（推断）：

- **8B**：DP=4。
- **32B**（BF16 65.5 GB）：吞吐优先用 FP8 + DP=4；保持 BF16 时用 TP2×DP2；对延迟极敏感时用 TP4。
- **70B 级**：BF16 用 TP4，或 FP8 + TP2×DP2。
- **30B-A3B**：以 DP=4 为基线，再试 DP4+EP。注意它只有 4 个 KV 头，TP4 时每卡只剩 1 个头。

DeepSeek 的大规模部署可以作为参照，但两份文档的配置不同，引用时要注明出处：

- V3 技术报告：decode 用 40 节点、EP320（arXiv 2412.19437）。
- Day 6 公开数据：decode 用 18 节点、EP144；每台 H800 节点平均约 73.7k tok/s 的 prefill 输入，或约 14.8k tok/s 的 decode 输出（open-infra-index Day 6）。

Hopper 上的量化（quantization README @v0.31.0、quantized_kvcache.md @v0.31.0）：

- Tensor Core 原生支持 FP8 和 INT8。FP4 和 MXFP8 属于 Blackwell 特性；这一点的依据是间接证据。
- **FP8 W8A8**：decode 时读取量减半，prefill 时算力翻倍，两头兼顾。
- **W4A16（Marlin）**：小 batch 时最快，但在 batch 约 74 时就进入 compute 区（推断）。Marlin 论文称 batch 16–32 时接近 4× 加速（arXiv 2408.11743）。
- **FP8 KV cache**：不做校准时 scale 默认为 1.0，官方推荐用数据集校准。

关于 FP8 累加精度，两份 DeepSeek 文档的说法互相矛盾：V3 报告说只用“highest 14 bits”，ISCA 论文说“13 fraction bits”，这里两种说法都保留。

关键 kernel：

- **FA3**：Hopper 专用，FP16 下达到 740 TFLOPs/s（75% 利用率），README 标为 beta（arXiv 2407.08608、flash-attention README）。
- **Flash-Decoding**：先把 KV 切块并行计算，再用 log-sum-exp 归约，解决 batch=1 时 GPU 只能用到不足 1% 的问题（CRFM blog）。
- **FlashInfer**：已被 vLLM 和 SGLang 采用（arXiv 2501.01005）。

profiling 的顺序：先用 nsys 看整体 timeline，判断有没有空泡、各类 kernel 的占比、NCCL 是否和计算重叠；再用 torch profiler 定位到 Python 调用栈；最后只对单个热点 kernel 用 Nsight Compute 深挖。Nsight Compute 的 CLI 语法「未核实」，今天先不做。

当日阅读：

- Megatron 第 3 节；nccl-tests PERFORMANCE.md。
- DeepSeek ISCA’25 论文中关于 H800 互联的部分。
- CS336 的 L7/L8 “Parallelism”。

实验与代码

实验 6-1：拓扑与 NCCL 带宽（4 卡，约 30 分钟）。

```
nvidia-smi topo -m
git clone https://github.com/NVIDIA/nccl-tests.git && cd nccl-tests && make
./build/all_reduce_perf -b 8 -e 8G -f 2 -g 4
./build/all_reduce_perf -b 8 -e 8G -f 2 -g 2
```

如果 make 找不到 CUDA 或 NCCL 的头文件，按 README 传入 CUDA_HOME 和 NCCL_HOME（nccl-tests README）。

实验 6-2：lab07 的 NCCL 版本（4 卡，约 30 分钟；本机未运行）。本机上 torchrun 因为 Windows 的 libuv 问题无法使用，只用 --spawn 2 --backend gloo 验证过正确性；Linux 上应该可以正常使用 torchrun。

```
torchrun --nproc_per_node=4 lab07_tensor_parallel.py --backend nccl --test \
    --max-mb 1024
```

lab07 把一个 Transformer 块的 TP 写成了不到 20 行：列并行的部分本地算完，行并行的部分各自得到部分和，再用 2 次 all_reduce 相加（摘自../labs/lab07_tensor_parallel.py）：

```
def block(x: Tensor, w: W, H: int, KV: int, D: int, reduce: bool) -> Tensor:
    """x: [T, dim]。H/KV 为本 rank 持有的头数；reduce=False 即单进程参考实现。"""
    T = x.shape[0]
    h = rms(x)
    q = (h @ w["wq"].T).view(T, H, D).transpose(0, 1)
    k = (h @ w["wk"].T).view(T, KV, D).transpose(0, 1).repeat_interleave(H // KV, 0)
    v = (h @ w["wv"].T).view(T, KV, D).transpose(0, 1).repeat_interleave(H // KV, 0)
    a = F.scaled_dot_product_attention(q, k, v, is_causal=True)  # 本地头，无通信
    o = a.transpose(0, 1).reshape(T, H * D) @ w["wo"].T            # 部分和
    if reduce:
        dist.all_reduce(o)                                         # 通信 #1
    x = x + o
    h = rms(x)
    m = (F.silu(h @ w["w_gate"].T) * (h @ w["w_up"].T)) @ w["w_down"].T
    if reduce:
        dist.all_reduce(m)                                         # 通信 #2
    return x + m
```

实验 6-3：8B 模型的 TP4 与 DP4（4 卡，约 3 小时）。

```
vllm serve Qwen/Qwen3-8B -tp=4 --port 8000
ps -ef | grep -i vllm   # 预期 1 API + 1 core + 4 worker = 6
vllm serve Qwen/Qwen3-8B -dp=4 --port 8000
ps -ef | grep -i vllm   # 按公式推算 4 API + 4 core + 4 worker + 1 协调器
```

在两种配置下都用 Day 5 的 open-loop 扫描（--request-rate 2..32 加 --goodput ttft:500 tpot:50）。另外测一组并发 1 的 TPOT。

实验 6-4：32B 与 30B-A3B 的并行选型（4 卡，约 6 小时）。

```
vllm serve Qwen/Qwen3-32B --tensor-parallel-size 4 --port 8000
vllm serve Qwen/Qwen3-32B --data-parallel-size 2 --tensor-parallel-size 2 \
    --port 8000
vllm serve Qwen/Qwen3-30B-A3B --data-parallel-size 4 --port 8000
vllm serve Qwen/Qwen3-30B-A3B --data-parallel-size 4 \
    --enable-expert-parallel --port 8000
vllm serve Qwen/Qwen3-30B-A3B --tensor-parallel-size 4 --port 8000
```

每种配置各测两档并发：并发 1 测单流 TPOT，并发 128 测吞吐。同时记录启动日志里的 KV 容量。

实验 6-5：nsys 抓 timeline（4 卡，约 2 小时）。对 32B TP4 配置运行 nsys；参数取自 vLLM profiling 文档：

```
VLLM_WORKER_MULTIPROC_METHOD=spawn nsys profile \
  --trace-fork-before-exec=true --cuda-graph-trace=node \
  --capture-range=cudaProfilerApi --capture-range-end repeat \
```

```
  vllm serve Qwen/Qwen3-32B --tensor-parallel-size 4 \
  --profiler-config.profiler cuda
# 另一终端：vllm bench serve ... --profile 少量采样；结束后：
nsys stats report1.nsys-rep
```

在 cuda_gpu_kern_sum 汇总表中找出这三类 kernel：ncclDevKernel_AllReduce、sm90_xmma_gemm_bf16…和 FlashAttnFwdSm90。

lab09 的 NVTX 标注方式可以直接用在你自己的脚本里：一处标注，torch profiler 和 nsys 都能看到（摘自../labs/lab09_profile.py）：

```
@contextmanager
def region(name: str, dev: torch.device) -> Iterator[None]:
    """record_function + NVTX：一处标注，两种工具都能看到。"""
    if dev.type == "cuda":
        torch.cuda.nvtx.range_push(name)
    try:
        with record_function(name):
            yield
    finally:
        if dev.type == "cuda":
            torch.cuda.nvtx.range_pop()
```

```
nsys profile -t cuda,nvtx,osrt -o out/lab09_nsys \
    python lab09_profile.py --device cuda --no-torch-profiler --dim 2048 --layers 16
```

这条命令来自 lab09 的 docstring，本机未运行。

实验 6-6：FP8 KV 的容量（1 卡，约 1 小时）。在 Python API 中用 LLM(model="Qwen/Qwen3-8B", kv_cache_dtype="fp8") 启动，对比启动日志中的 KV token 数。注意这是没做校准（scale=1.0）的版本，**只能用来看容量，不能据此判断精度。**

OxyGen 对照：小模型多加卡不降时延

OxyGen v3 报告：TP=1 时加速比为 3.50×，TP=2 时为 3.71×，但开启 TP=2 后，帧时延反而从 493.4 ms 升到 535.3 ms，两块 GPU 并没有降低时延（OxyGen v3）。这与”单卡放得下就优先 DP”的规则一致：2B 级模型的 TP 通信和同步成本，超过了计算切分带来的收益。

可选实验（GPU 空闲时）：OxyGen 的 E6 和 E7。

- **E6**：用 profile_sample_actions 对 batch 1–32 计时，找出去噪从 launch/memory-bound 转为 compute-bound 的拐点。参考数据：PhyAI 在 Hopper 上测得，batch=1 时 action expert 只占 8.8% 的 FLOPs，却占 57.2% 的时延（arXiv 2608.03682）。
- **E7**：每张卡起一个独立的 OxyGen 实例，模拟 4 台机器人，这就是 DP；再与单卡 MPS 的结果对比。

验收标准与资源

验收标准：

- [概念] 推出 TP 每层 2 次 all-reduce、单次消息为 tokens×h×2 B；把实测的 busbw 代回去，重算 8192 token prefill 的通信时间；能解释为什么不能直接用 FlashMLA 的 main 分支。
- [实验] 给出 busbw 曲线，以及 8 KiB、512 KiB、64 MiB 三个消息大小的耗时。lab07 NCCL 版 2/2 PASS，它的 busbw 与 nccl-tests 在同一量级。
- [实验] 进程数与公式一致；画出 8B 模型 TP4 与 DP4 的 rate–goodput 曲线，写出结论。
- [实验] 32B 和 30B-A3B 每种配置在两档并发下的表格，最后给出选型建议。
- [实验] nsys 中 NCCL kernel 占 GPU 时间的百分比，并与上面的估算对照。
- [实验] FP8 KV 的 token 容量约为 BF16 的 2 倍。

资源：4 卡整机，约 20 GPU·h。新增模型 Qwen3-32B（约 65.5 GB）和 Qwen3-30B-A3B（约 61 GB）。

Day 7：像素、帧和波形最终都变成 prefill token

目标：能手算图像、视频和音频输入的 token 数及对应的 KV 占用；在 vLLM 上测出”视觉 token 数 →TTFT”的关系和多模态缓存的效果；理解 Omni 多阶段流水线和扩散生成在 Infra 上的不同；最后把 OxyGen 用这一周学到的语言完整重述一遍。

理论：VLM、视频、音频、Omni、扩散与 VLA

VLM 的通路分四步：

1. CPU 预处理：解码、resize、归一化、切 patch。
2. ViT 类视觉编码器（在 GPU 上）。
3. merger/projector：把视觉特征映射到 LLM 的 hidden size，同时压缩 token 数。
4. 在文本中预留同样数量的占位 token，用 `masked_scatter` 把视觉 embedding 写进去。

以 Qwen3-VL 为例，`num_image_tokens = image_grid_thw.prod() // merge_size**2`（processing_qwen3_vl.py @v5.19.0）。视觉 token 一旦写进序列，占用的 KV 与文本 token 完全一样。

几种常见模型的 token 数规则：

- **Qwen2/2.5-VL**：`smart_resize` 把边长取整到 28 的倍数，每个 LLM 视觉 token 对应 28×28 个像素，模型仓库配置的上限是 16,384 token（image_processing_qwen2_vl.py @v5.19.0、Qwen2.5-VL-7B preprocessor_config.json）。
- **Qwen3-VL**：patch 是 16，所以每 token 对应 32×32 个像素。
- **LLaVA-1.5**：固定 576 个 token。
- **Gemma 3**：每张图固定 256 个 token。

1920×1080 输入的手算结果（推断，公式见上述源码）：

模型	每 token 像素	1080p 图 token 数	单图 KV（bf16）
Qwen2.5-VL-7B	28×28	2,691	约 147 MiB
Qwen3-VL-8B	32×32	2,040	约 287 MiB
LLaVA-1.5	固定	576	—
Gemma 3	固定	256（P&S 最多 1,280）	—

两点需要注意：

- 归一化常数各不相同：Qwen2.5-VL和 LLaVA 用 OpenAI CLIP 的均值方差，Qwen3-VL 和 Gemma 3 用 0.5/0.5，InternVL 用 ImageNet 的。自己写预处理时用错常数，精度会静默下降。
- transformers v5.19.0 已把向 processor 传 `min_pixels/max_pixels` 标为弃用（计划在 v5.23 移除），新的写法是 `size={'longest_edge':…, 'shortest_edge':…}`。

视频的 token 爆炸。 qwen-vl-utils 的默认规则是（qwen-vl-utils vision_process.py）：

- 按 2 fps 采样，帧数限制在 [4, 768]。
- 相邻 2 帧合成一个 temporal patch。
- 每个 temporal patch 的 token 数在 128–768 之间。
- 整段视频的总预算约为 0.9×128000 个 token。

推算两个例子（推断）：

- 60 秒、30 fps、1080p 的视频，在 Qwen2.5-VL 中约 **43,200 token**，KV 约 **2.31 GiB**。
- 10 分钟的视频会顶到约 115,200 token 的预算上限。

transformers v5.19.0 的 Qwen3-VL 视频处理器默认**不对每帧像素封顶**，必须设 `cap_pixels_per_frame` 才能复现 qwen-vl-utils 的结果；这个默认值计划在 v5.22 改变（video_processing_qwen3_vl.py @v5.19.0）。vLLM 提供 `video_pruning_rate`，默认方法为 EVS（multimodal.py @v0.31.0）。

音频。各前端的 token 率：

- **Whisper**：16 kHz 采样，n_fft=400，hop=160；30 秒切一块，得到 3000 帧 Mel；conv2 的 stride 为 2，所以是 **1500 个编码器位置**（每秒 50 个）。输入长度固定为 30 秒，短音频也要 pad 到 30 秒（whisper/audio.py、whisper/model.py）。
- **Qwen2-Audio 和 Qwen2.5-Omni**：在编码器后再做一次 stride-2 池化，得到 **25 token/s**，30 秒对应 750 个 token（modeling_qwen2_audio.py @v5.19.0）。
- **Qwen3-Omni 的 AuT 编码器**：8 倍下采样，得到 **12.5 Hz。**实现上每 1 秒的块产出 13 个 token（Qwen3-Omni report arXiv 2509.17765）。

Omni 与语音输出。 Qwen3-Omni 是三段流水线：Thinker（MoE）、Talker（MoE，每步输出 1 帧 12.5 Hz 的 codec，再用 MTP 补齐其余 15 个码本）、Code2Wav（轻量因果 ConvNet）。报告给出的冷启动理论首包延迟为 **234 ms。**

做 serving 时，三个阶段各有自己的 KV 或状态，阶段之间是流式的生产者-消费者关系，需要分别调度。这和 OxyGen”一次 prefill 结果被多个专家消费”是同一类问题（推断）。vLLM 组织下有一个独立项目 vLLM-Omni，最新版本 v0.30.0；它的 CLI 和支持的模型列表「未核实」（vLLM-Omni v0.30.0）。

扩散生成（下面整段都是通用知识，研究笔记没有抓到一手来源，**「未核实」，不能当作已验证的事实**）：

- 推理流程：文本编码一次；DiT/UNet去噪 N 步，每步一次完整前向；开 CFG 时每步要算条件和无条件两次；最后 VAE 解码一次。
- 没有随长度增长的 KV cache，瓶颈是”步数 × 每步 token 数 ^{2}”的注意力计算。
- 优化方向：减少步数、跨步缓存特征（TeaCache等）、序列并行（xDiT 的 USP）。

能核实的只有版本：diffusers v0.41.0、xDiT 0.7.0，以及 SGLang v0.5.21 源码树中存在 `python/sglang/multimodal_gen/` 目录（diffusers v0.41.0、xDiT 0.7.0、sglang multimodal_gen）。DiT 论文见 arXiv 2212.09748。扩散生成的实验放到第二周。

VLA。 π0/π0.5 的推理就是一个 VLM 的 prefill（SigLIP So400m/14 把每张图编码为 256 个 token，再与文本一起送进 Gemma-2B），后面接 10 步 Euler flow matching。每一步只把 H 个动作 token 送进约 300M 的 action expert，读取前缀 KV，但不追加 KV（openpi pi0.py、π0 arXiv 2410.24164）。

开源的 openpi 中，π0.5 只支持 flow matching 头。π0.5 论文里”先解码子任务文本，再出动作”的那条路径没有开源，OxyGen 的语言路径来自社区复现（openpi README）。

对比 π0-FAST：它用完整的 2B 骨干自回归解码 FAST token，在 4090 上每个动作块约 750 ms，而 π0 不到 100 ms（FAST arXiv 2501.09747）。这个差距本身就是 Day 2”decode 受带宽限制”的现实写照。

当日阅读：

- 源码：Qwen3-VL的 processing 和 modeling 文件。
- 论文：π0.5 arXiv 2504.16054。
- 选读：Beyond LLM Serving arXiv 2610.05062，它指出单机器人 batch-1 推理不在 LLM serving 系统的设计点上。

实验与代码

实验 7-1：玩具多模态通路（CPU，1 小时）。

```
python lab10_multimodal_toy.py --device cpu --test     # 本机 11/11 PASS
python lab10_multimodal_toy.py --image-hw 448 672 --patch 14 --n-mels 128
```

第二条命令的参数组合本机没有单独运行过。在 128 个 Mel 滤波器下，应该能观察到 `empty mel filters=4`。

lab10 的核心代码：卷积 patch 嵌入，2×2 合并，投影，然后按占位符的顺序写入文本序列（摘自../labs/lab10_multimodal_toy.py）：

```
    with torch.no_grad():
        emb_conv = conv(x[None]).flatten(2)[0].T              # [N, vit_dim]
```

```
        emb_lin = patches @ conv.weight.reshape(a.vit_dim, -1).T + conv.bias
        merged = merge2x2(emb_lin, gh, gw)                      # [N/4, 4·vit_dim]
        proj = nn.Sequential(nn.Linear(4 * a.vit_dim, a.llm_dim), nn.GELU(),
                             nn.Linear(a.llm_dim, a.llm_dim)).to(dev)
        vis = proj(merged)                                      # [M, llm_dim]
        M, IMG = vis.shape[0], a.vocab - 1              # 最后一个 id 当图像占位符
        ids = torch.cat([torch.randint(0, IMG, (5,)), torch.full((M,), IMG),
                         torch.randint(0, IMG, (7,))]).to(dev)
        tok_emb = nn.Embedding(a.vocab, a.llm_dim).to(dev)
        mask = ids == IMG
        seq = tok_emb(ids).masked_scatter(mask[:, None], vis)  # 按顺序填入占位符位置
```

实验 7-2：真实 processor 的 token 计数（CPU，1 小时，在 transformers 的 venv 中做）。

1. 合成 256^2、1280×720、1920×1080、3840×2160 四张图。
2. 分别用 Qwen2.5-VL-7B 和 Qwen3-VL-8B 的 `AutoProcessor` 处理，读出 `image_grid_thw`，计算 `prod//4`。
3. 用 LLaVA-1.5 统计 `input_ids` 中图像 token 的个数。
4. 视频部分：分别设置 `cap_pixels_per_frame=True` 和 `False`，比较 `video_grid_thw`。

调用细节以当版文档为准。

实验 7-3：分辨率扫描 TTFT（GPU0，约 2 小时）。

```
CUDA_VISIBLE_DEVICES=0 vllm serve Qwen/Qwen2.5-VL-7B-Instruct --port 8000 \
  --limit-mm-per-prompt '{"image": 4, "video": 1}' \
  --mm-processor-kwargs \
  '{"size": {"longest_edge": 12845056, "shortest_edge": 3136}}' \
  --max-num-batched-tokens 16384
```

用 `/v1/chat/completions` 发送 `image_url` 消息，设 `max_tokens=1`，分辨率从 448^2 一直扫到 4K，测出 TTFT。然后把 `longest_edge` 改成 `1003520`（即 1280×28×28）重测一遍。

实验 7-4：多模态缓存（GPU1，约 1 小时）。同一张 1080p 图配不同的问题，连续发送 50 次，分三组：

- A 组：默认配置（processor cache 4 GiB，prefix caching 开启）。
- B 组：加 `--mm-processor-cache-gb 0`。
- C 组（可选）：加 `--no-enable-prefix-caching`。

实验 7-5（可选，4 卡，约 3 小时）：encoder TP 模式与视频剪枝。对比 `vllm serve Qwen/Qwen2.5-VL-72B-Instruct --tensor-parallel-size 4 --mm-encoder-tp-mode data` 和 `weights` 两种模式；在 60 秒视频上再对比加上 `--video-pruning-rate 0.5` 的效果。注意 72B 需要额外下载约 145 GB，而且它是否支持 `data` 模式「未核实」（不支持时会自动回退）。

实验 7-6：OxyGen 的总复盘（GPU2 和 GPU3，约 3 小时）。在 H800 上跑完整的四种设置：

```
uv run python -m experiments.run_experiments \
  --settings baseline shared_kv continuous_batching parallel_mps \
  --policies pi05_o2_aloha pi05_o2_droid pi05_o2_libero \
  --checkpoint-dir /tmp/oxygen_random_init_checkpoint \
  --results-dir experiments/results_h800_jax --gpu 2
```

把结果与论文中 RTX 4090 的数据对照：

- 消融：仅开 KV sharing 时，N=5 的帧时延从 200.3 ms 降到 145.1 ms；再开 batching，N=30、k=5 时从 469.3 ms 降到 175.8 ms。
- 人形机器人实测：在 Thor 上，isolated 执行为 1030 ms，OxyGen 为 393 ms（OxyGen v3）。

OxyGen 对照：把你的系统写成一页 Infra 说明书

今天的收尾作业是用这一周的术语重新描述 OxyGen：

- **预处理**：图像缩放到 224；状态离散化后写进 prompt。
- **prefill**：3×256 个视觉 token 加文本 token；Gemma-2B MQA，每 token 18 KiB 的 KV。
- **两个 KV 消费者**：flow-matching 专家，S 次只追加 query、不追加 KV 的小前向；自回归语言专家，带帧粒度的 continuous batching。
- **SLO**：动作有逐帧截止期，语言是软截止期。
- **局限**：k固定，没有自适应；语言输出基于"陈旧观测"；KV 静态预留，不分页。

最后，说出 OxyGen 和 Omni 的 Thinker-Talker、Robion 的双 stream 解耦分别有什么异同（Robion arXiv 2609.12075）。

验收标准与资源

验收标准：

- [概念] 手算出表中 2,691、2,040、576 三个数，以及 60 秒视频的 43,200 token；说出 Whisper 为什么固定 1500 个位置；说出 Qwen3-Omni 首包延迟由哪几段组成。
- [实验] lab10 11/11 PASS；真实 processor 的计数与手算误差为 0（公式是确定性的）。
- [实验] 画出 TTFT–视觉 token 数曲线，并用 56 KiB/token 去核对 KV 使用量的变化。
- [实验] 缓存实验给出首请求与后续请求的 TTFT 比值，以及 A/B 两组 API 进程的 CPU 时间差。
- [实验] OxyGen 在 H800 上四种设置的帧时延和语言吞吐表，与 4090 的数据对照，并写出差异的原因(推断要标注清楚)。
- [概念] 产出"一页纸"：请求生命周期图、KV 和 roofline 手算表、OxyGen 重述。

资源：约 10 GPU·h（可选的 72B 实验另计约 12 GPU·h）。模型为 Qwen2.5-VL-7B 和 Qwen3-VL-8B。

自检题：每一题都应能用数字或代码回答

Day 1：

1. Qwen3 config 的 `vocab_size` 是 151,936，而 tokenizer 只有 151,669 个 token，多出来的行是什么?
2. 为什么 HF 静态 batch 生成要用 left padding，而 vLLM 不需要?
3. "龘"被切成两个 token，单独 decode 第一个 token 会得到什么？流式输出应该怎么处理?
4. 为什么 RoPE 只作用于 Q 和 K？缓存里存的是旋转前的 K，还是旋转后的?
5. HF API 报告 Qwen3-0.6B 有 751,632,384 个参数，但按 `tie_word_embeddings=true` 计算只有约 0.596B。差额约等于一个 151,936×1024 的 embedding 表，可能的原因是什么？(推断：safetensors 里单独存了 lm_head，「未核实」)

Day 2：

6. 写出 KV/token 公式，分别算出 Qwen3-8B、Qwen3-30B-A3B 和 π0.5 Gemma-2B 的值。(144 KiB、96 KiB、18,432 B)
7. Qwen3-8B 在单卡 H800、batch=1 时的 decode 带宽下限是多少？lab08 算出约 5.1 ms，与 4.5 ms 的差异来自哪里?
8. 为什么 decode attention 的算术强度约等于 Hq/Hkv，并且与 batch 无关?
9. vLLM 在 0.92 利用率下是怎样得出 KV 容量的？扣除了哪几项?
10. DynamicCache 为什么和 torch.compile、CUDA graph 不兼容?

Day 3：

11. logits 为 [2, 1, 0.5, −1]，T=0.5、top-p=0.9，最后保留哪些 token，概率各是多少?
12. vLLM 默认返回的 logprobs 是在采样流程的哪一步计算的?
13. temperature=0 为什么仍然可能不确定？batch invariance 解决的是什么问题?
14. stop 字符串为什么必须在 detokenizer 层缓冲?

Day 4：

15. vLLM V1 调度器为什么说自己"没有 prefill 阶段和 decode 阶段"?
16. block_size=16 时，一个 1,000 token 的 prompt 有几块能进入 prefix cache？(62 块)

17. 为什么请求结束时要逆序释放它的块?
18. vLLM V1 抢占一个请求后，它的 KV 去了哪里？代价是什么?
19. OxyGen 在 uniform 到达、N=30、k=5 时，稳态 batch 约为多少?

Day 5：

20. 写出 TTFT、ITL、TPOT、goodput 的定义。open-loop 和 closed-loop 的压测结果能直接比较吗?
21. `vllm serve -tp=2 -dp=4` 会启动多少个进程?
22. speculative decoding 为什么能保证输出分布不变？什么情况下收益会下降?
23. 什么时候选 PD 分离，什么时候选 chunked prefill?
24. 为什么 OxyGen 的 action frequency = H/T 不是反应时延?

Day 6：

25. Qwen3-8B 用 TP4 做 8192 token 的 prefill，单次 all-reduce 的消息多大？每步总共做几次?
26. busbw 和 algbw 怎么换算？为什么 busbw 可以直接和硬件峰值比较?
27. 在 4×H800 上，为什么 8B 模型优先用 DP=4？32B 有哪三种选择?
28. FP8 KV cache 不做校准时 scale 是多少？为什么不能只看 perplexity?
29. 为什么 H800 用户必须把 FlashMLA 固定在 `ba89a34`?

Day 7：

30. 1080p 图像在 Qwen2.5-VL、Qwen3-VL-8B 和 LLaVA-1.5 中各变成多少个 token?
31. 60 秒 1080p 视频在 Qwen2.5-VL 默认设置下约有多少 token、占多少 KV?
32. Whisper 为什么总是 1500 个编码器位置？Qwen3-Omni 的音频 token 率是多少?
33. 为什么 π0.5 的前缀 KV 不能像 LLM prefix caching 那样跨帧精确复用?
34. flow matching 的去噪步和 LLM 的 decode 步，在 Infra 上有哪三点不同?

结论

这一周最值得带走的，不是某个参数的默认值，而是一个判断框架：**推理 Infra 是在 SLO 约束下分配两种稀缺资源——HBM 的带宽和容量（权重加 KV），以及算力。**用这个框架看前面学过的内容：

- PagedAttention 和 prefix caching 解决”容量”问题。
- continuous batching、投机解码和量化，都在摊薄”每步读一遍权重”的带宽成本。
- chunked prefill、PD 分离和 OxyGen 的固定 k，都在用”每轮工作量上限”或”物理隔离”去保护时延敏感的那一路。
- 并行策略则要在通信带宽这第三种资源上做取舍；H800的 400 GB/s NVLink 让这个取舍明显偏向 DP。

一旦你能把任何一项新技术放进”它省了哪种资源、牺牲了哪个指标”这个格子里，就算真正入门了。

对你来说，下一步最自然的延伸来自 OxyGen 自己承认的局限：k 是固定的。用 Day 5 的 slack 测量去驱动一个自适应的 k 控制器，本质上就是把 vLLM 的自适应 `long_prefill_token_threshold` 和 SLO 驱动的调度思想，移植到帧粒度的 VLA 场景中去。这也是从”可运行实验”走向”工程调优”、乃至研究贡献的一个具体抓手。

生产级能力则要在第二周以后，用真实流量和集群去补：容量规划、告警、扩缩容、故障迁移和回归基准。这几样无法在单机七天内伪造出来。

参考资料

以下事实均来自研究笔记中 2026-10-14/15 访问的来源。版本敏感的条目以对应 tag 为准。

推理引擎与调度：

- vLLM v0.31.0 release —GitHub；PyPI —vllm 0.31.0
- vLLM 架构总览—arch_overview.md @v0.31.0
- vLLM 调度器—scheduler.py @v0.31.0；config/scheduler.py

- vLLM 缓存配置与 prefix caching —cache.py；prefix_caching.md
- vLLM 参数与默认值—arg_utils.py @v0.31.0；vllm.py
- vLLM MRV2 与 CUDA graph —model_runner_v2.md；cuda_graphs.md
- vLLM 显存—gpu_worker.py；conserving_memory.md
- vLLM 压测与指标—benchmarks/serve.py；loggers.py；metrics design
- vLLM 采样—sampler.py @v0.31.0；sampling_params.py
- vLLM 确定性—batch_invariance.md；reproducibility.md
- vLLM 结构化输出—structured_outputs.md @v0.31.0
- vLLM 安装（冲突来源）—gpu.cuda.inc.md @v0.31.0；最新安装文档
- vLLM 并行—parallelism_scaling.md；data_parallel_deployment.md；expert_parallel_deployment.md
- vLLM profiling 与量化—profiling.md；quantization README；quantized_kvcache.md
- vLLM 多模态配置—multimodal.py @v0.31.0
- SGLang v0.5.21 —release；install.mdx；PyPI
- SGLang 调度与缓存—scheduler.py；schedule.py；radix_cache.py
- SGLang 服务化—pd_disaggregation.mdx；sgl_model_gateway.mdx；benchmark/serving.py；production_metrics.mdx

奠基论文：

- Orca —USENIX OSDI’22；PagedAttention —arXiv 2309.06180
- SARATHI —2308.16369；Sarathi-Serve —2403.02310；SGLang —2312.07104
- DistServe —2401.09670；Splitwise —2311.18677；Mooncake —2407.00079
- Speculative decoding —Leviathan 2211.17192；Chen 2302.01318；Medusa 2401.10774；EAGLE 2401.15077；EAGLE-3 2503.01840
- Pope et al. —2211.05102；MQA —1911.02150；GQA —2305.13245
- Transformer —1706.03762；RoPE —2104.09864；RMSNorm —1910.07467；SwiGLU —2002.05202
- DeepSeek-V2 —2405.04434；DeepSeek-V3 —2412.19437；Insights into DeepSeek-V3 —2505.09343
- FlashAttention —2205.14135；FA2 —2307.08691；FA3 —2407.08608；FA4 —2603.05451
- Flash-Decoding —CRFM blog；FlashInfer —2501.01005
- Megatron-LM —1909.08053；Context Parallelism —2411.01783
- 量化—FP8 Formats 2209.05433；SmoothQuant 2211.10438；AWQ 2306.00978；GPTQ 2210.17323；Marlin 2408.11743
- 采样与约束解码—Holtzman 1904.09751；min-p 2407.01082；Outlines 2307.09702；XGrammar 2411.15100；llguidance
- 非确定性—Thinking Machines blog
- 多实例调度—Llumnix 2406.03243

模型、tokenizer 与 HF 实现：

- Qwen3-0.6B —tokenizer.json；tokenizer_config.json；generation_config.json
- Qwen3 configs —8B；32B；30B-A3B
- Qwen 文档—Key Concepts；SentencePiece —README
- transformers v5.19.0 源码—modeling_qwen3.py；cache_utils.py
- transformers 文档与生成代码—kv_cache.md；cache_explanation.md；llm_tutorial.md；streamers.py；generation/utils.py

多模态：

- Qwen-VL 系列—processing_qwen3_vl.py；modeling_qwen3_vl.py；image_processing_qwen2_vl.py；video_processing_qwen3_vl.py
- Qwen-VL 配置与工具—Qwen2.5-VL-7B preprocessor_config；qwen-vl-utils
- 其他 VLM —Gemma 3 report 2503.19786；LLaVA-1.5 config
- 音频—whisper audio.py；whisper model.py；Qwen2-Audio modeling
- Omni 与服务框架—Qwen3-Omni report 2509.17765；vLLM-Omni v0.30.0

- 扩散—diffusers v0.41.0；xDiT 0.7.0；DiT 2212.09748

VLA 与 OxyGen：

- OxyGen 论文—arXiv 2603.14371；v3 HTML
- OxyGen 代码 @1e88906 —README；Experiments.md；policy.py；pi05.py；kv_cache_manager.py
- openpi @215abfb —pi0.py；gemma.py；README
- π 系列论文—π0 2410.24164；π0.5 2504.16054；FAST 2501.09747；PaliGemma 2407.07726
- 相关系统—realtime-vla 2510.26742；PhyAI 2608.03682；Beyond LLM Serving 2610.05062；VLA-Cache 2502.02175；Robion 2609.12075；Kairos 2605.11381

硬件、环境与工具：

- NVIDIA —H100 产品页；Hopper 架构博客；H800 页面（404）
- H800 二手来源—腾讯云规格；Lenovo LP1814；强川科技（T3）
- CUDA 与 nvidia-smi —CUDA Toolkit Release Notes；nvidia-smi 文档
- NCCL 与 profiling —nccl-tests；PERFORMANCE.md；Nsight Systems User Guide
- DeepSeek 开源组件—FlashMLA；FlashMLA @ba89a34；DeepGEMM；DeepEP；Day 6 推理系统
- 下载工具—HF CLI guide

学习资源与生产化：

- 代码仓库—nano-vllm；mini-sglang；gpt-fast；minbpe；nanochat
- 课程—CS336；CMU 11-868；MIT 6.5940；GPU MODE lectures；Awesome-ML-SYS-Tutorial
- 手册与文档—HF llm_optims；LLM Inference Handbook
- 生产部署—GKE autoscaling；llm-d；Dynamo；KServe；production-stack

附录：代码仓库、文件与工具链接清单

本附录把正文和参考资料中用到的链接以完整 URL 列出，方便直接复制。核实日期 2026-10-15；源码文件链接固定在对应 tag 或 commit 上。配套的环境搭建操作手册见`../AGENT_SETUP.md`，脚本见`../scripts/`，代码实验见`../labs/`。

推理引擎仓库与官方文档

- vLLM 仓库：https://github.com/vllm-project/vllm
- vLLM v0.31.0 发布说明：https://github.com/vllm-project/vllm/releases/tag/v0.31.0
- vLLM 文档首页：https://docs.vllm.ai/
- vLLM GPU 安装：https://docs.vllm.ai/en/latest/getting_started/installation/gpu.html
- vLLM Docker 部署：https://docs.vllm.ai/en/latest/deployment/docker.html
- vLLM 压测 CLI：https://docs.vllm.ai/en/latest/benchmarking/cli/
- SGLang 仓库：https://github.com/sgl-project/sglang
- SGLang v0.5.21 发布说明：https://github.com/sgl-project/sglang/releases/tag/v0.5.21
- SGLang 文档首页：https://docs.sglang.io/
- SGLang 安装：https://docs.sglang.io/get_started/install.html
- vLLM-Omni：https://github.com/vllm-project/vllm-omni
- llm-d：https://github.com/llm-d/llm-d
- NVIDIA Dynamo：https://github.com/ai-dynamo/dynamo
- KServe：https://github.com/kserve/kserve
- vLLM production-stack：https://github.com/vllm-project/production-stack

关键源码与设计文档（按阅读顺序）

vLLM（tag v0.31.0）：

- 架构总览：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/design/arch_overview.md
- 调度器：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/core/sched/scheduler.py

- KV cache 管理：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/core/kv_cache_manager.py
- 块池：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/core/block_pool.py
- 前缀缓存设计：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/design/prefix_caching.md
- EngineCore 主循环：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/engine/core.py
- AsyncLLM 前端：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/engine/async_llm.py
- Model Runner（V1）：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/worker/gpu_model_runner.py
- Model Runner V2：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/worker/gpu/model_runner.py
- MRV2 设计文档：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/design/model_runner_v2.md
- CUDA Graph 设计：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/design/cuda_graphs.md
- 采样器：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/sample/sampler.py
- 引擎参数与默认值：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/engine/arg_utils.py
- 调度配置：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/config/scheduler.py
- 缓存配置：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/config/cache.py
- 显存与 worker：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/v1/worker/gpu_worker.py
- 压测实现：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/benchmarks/serve.py
- 指标说明：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/usage/metrics.md
- Profiling：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/contributing/profiling.md
- 并行部署：https://github.com/vllm-project/vllm/blob/v0.31.0/docs/serving/parallelism_scaling.md
- 多模态配置：https://github.com/vllm-project/vllm/blob/v0.31.0/vllm/config/multimodal.py

SGLang（tag v0.5.21）：

- 调度器：https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/managers/scheduler.py
- Radix Cache：https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/mem_cache/radix_cache.py
- 压测实现：https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/benchmark/serving.py
- PD 分离文档：https://github.com/sgl-project/sglang/blob/v0.5.21/docs/docs/advanced_features/pd_disaggregation.mdx
- Model Gateway 文档：https://github.com/sgl-project/sglang/blob/v0.5.21/docs/docs/advanced_features/sgl_model_gateway.mdx
- 指标文档：https://github.com/sgl-project/sglang/blob/v0.5.21/docs/docs/references/production_metrics.mdx

Hugging Face transformers（tag v5.19.0）：

- Qwen3 模型：https://github.com/huggingface/transformers/blob/v5.19.0/src/transformers/models/qwen3/modeling_qwen3.py
- Cache 实现：https://github.com/huggingface/transformers/blob/v5.19.0/src/transformers/cache_utils.py
- generate 实现：https://github.com/huggingface/transformers/blob/v5.19.0/src/transformers/generation/utils.py
- Qwen2-VL 图像预处理（smart_resize）：https://github.com/huggingface/transformers/blob/v5.19.0/src/transformers/models/qwen2_vl/image_processing_qwen2_vl.py
- Qwen3-VL 处理器：https://github.com/huggingface/transformers/blob/v5.19.0/src/transformer

s/models/qwen3_vl/processing_qwen3_vl.py
- Qwen3-VL 模型：https://github.com/huggingface/transformers/blob/v5.19.0/src/transformers/models/qwen3_vl/modeling_qwen3_vl.py

OxyGen（commit 1e88906）与 openpi（commit 215abfb）：

- OxyGen 仓库：https://github.com/air-embodied-brain/OxyGen
- OxyGen KV 管理：https://github.com/air-embodied-brain/OxyGen/blob/1e8890681090c7f423f85866b5db877916ba65e6/src/openpi/models/kv_cache_manager.py
- OxyGen π0.5 模型：https://github.com/air-embodied-brain/OxyGen/blob/1e8890681090c7f423f85866b5db877916ba65e6/src/openpi/models/pi05.py
- OxyGen 策略入口：https://github.com/air-embodied-brain/OxyGen/blob/1e8890681090c7f423f85866b5db877916ba65e6/src/openpi/policies/policy.py
- OxyGen 实验说明：https://github.com/air-embodied-brain/OxyGen/blob/1e8890681090c7f423f85866b5db877916ba65e6/experiments/Experiments.md
- openpi 仓库：https://github.com/Physical-Intelligence/openpi
- openpi π0 模型：https://github.com/Physical-Intelligence/openpi/blob/215abfb217dbac7d5f1273282331b9b1866c0479/src/openpi/models/pi0.py

其他源码与组件：

- Whisper 音频前处理：https://github.com/openai/whisper/blob/v20250625/whisper/audio.py
- Qwen-VL 视频/图像工具：https://github.com/QwenLM/Qwen3-VL/blob/main/qwen-vl-utils/src/qwen_vl_utils/vision_process.py
- DeepSeek 推理系统（Day 6）：https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md
- FlashMLA（最后支持 Hopper 的 commit）：https://github.com/deepseek-ai/FlashMLA/tree/ba89a3466e9470ad08ab39738d4e7bb66989e1e7
- DeepGEMM：https://github.com/deepseek-ai/DeepGEMM
- DeepEP：https://github.com/deepseek-ai/DeepEP
- FlashAttention：https://github.com/Dao-AILab/flash-attention
- FlashInfer：https://github.com/flashinfer-ai/flashinfer

学习用代码仓库

- nano-vllm（约 1.2k 行）：https://github.com/GeeeekExplorer/nano-vllm
- mini-sglang（约 5k 行）：https://github.com/sgl-project/mini-sglang
- gpt-fast：https://github.com/meta-pytorch/gpt-fast
- minbpe：https://github.com/karpathy/minbpe
- nanoGPT：https://github.com/karpathy/nanoGPT
- nanochat：https://github.com/karpathy/nanochat
- Awesome-ML-SYS-Tutorial（中文）：https://github.com/zhaochenyang20/Awesome-ML-SYS-Tutorial
- 本学习包代码实验：`../labs/`（说明见`../labs/README.md`）

工具

- uv：https://docs.astral.sh/uv/
- Hugging Face CLI：https://huggingface.co/docs/huggingface_hub/en/guides/cli
- nccl-tests：https://github.com/NVIDIA/nccl-tests
- nccl-tests 带宽说明：https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md
- Nsight Systems：https://developer.nvidia.com/nsight-systems
- Nsight Systems 用户手册：https://docs.nvidia.com/nsight-systems/UserGuide/index.html
- Nsight Compute：https://developer.nvidia.com/nsight-compute
- Perfetto（打开 trace）：https://ui.perfetto.dev

- nvidia-smi 文档：https://docs.nvidia.com/deploy/nvidia-smi/index.html
- CUDA 驱动版本对应：https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html
- vLLM Docker 镜像：https://hub.docker.com/r/vllm/vllm-openai
- SGLang Docker 镜像：https://hub.docker.com/r/lmsysorg/sglang

模型卡

- Qwen3-0.6B：https://huggingface.co/Qwen/Qwen3-0.6B
- Qwen3-8B：https://huggingface.co/Qwen/Qwen3-8B
- Qwen3-32B：https://huggingface.co/Qwen/Qwen3-32B
- Qwen3-30B-A3B：https://huggingface.co/Qwen/Qwen3-30B-A3B
- Qwen2.5-VL-7B-Instruct：https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct
- Qwen3-VL-8B-Instruct：https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct
- whisper-large-v3：https://huggingface.co/openai/whisper-large-v3
- Qwen2.5-Omni-7B：https://huggingface.co/Qwen/Qwen2.5-Omni-7B
- DeepSeek-V2-Lite-Chat：https://huggingface.co/deepseek-ai/DeepSeek-V2-Lite-Chat
- Llama-3.1-8B-Instruct（gated，需申请）：https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

数据集

- ShareGPT（vLLM 压测文档所用）：https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered
- 直接下载文件：https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/ShareGPT_V3_unfiltered_cleaned_split.json

课程与文档

- Stanford CS336：http://cs336.stanford.edu/
- CMU 11-868 LLM Systems：https://llmsystem.github.io/
- MIT 6.5940 EfficientML：https://hanlab.mit.edu/course
- GPU MODE 讲座：https://github.com/gpu-mode/lectures
- HF 推理优化文档：https://huggingface.co/docs/transformers/main/en/llm_optims
- LLM Inference Handbook：https://handbook.modular.com/

论文（建议先读前 8 篇）

1. PagedAttention / vLLM：https://arxiv.org/abs/2309.06180
2. Orca：https://www.usenix.org/conference/osdi22/presentation/yu
3. FlashAttention：https://arxiv.org/abs/2205.14135
4. Sarathi-Serve：https://arxiv.org/abs/2403.02310
5. DistServe：https://arxiv.org/abs/2401.09670
6. SGLang / RadixAttention：https://arxiv.org/abs/2312.07104
7. Speculative Decoding（Leviathan 等）：https://arxiv.org/abs/2211.17192
8. Mooncake：https://arxiv.org/abs/2407.00079

扩展阅读：

- Efficiently Scaling Transformer Inference（Pope 等）：https://arxiv.org/abs/2211.05102
- FlashAttention-2：https://arxiv.org/abs/2307.08691
- FlashAttention-3：https://arxiv.org/abs/2407.08608
- FlashInfer：https://arxiv.org/abs/2501.01005
- Splitwise：https://arxiv.org/abs/2311.18677
- Medusa：https://arxiv.org/abs/2401.10774
- EAGLE：https://arxiv.org/abs/2401.15077

- EAGLE-3：https://arxiv.org/abs/2503.01840
- Llumnix：https://arxiv.org/abs/2406.03243
- Megatron-LM：https://arxiv.org/abs/1909.08053
- DeepSeek-V3 技术报告：https://arxiv.org/abs/2412.19437
- XGrammar：https://arxiv.org/abs/2411.15100
- OxyGen：https://arxiv.org/abs/2603.14371
- π0：https://arxiv.org/abs/2410.24164
- π0.5：https://arxiv.org/abs/2504.16054
- Qwen3-Omni 技术报告：https://arxiv.org/abs/2509.17765